

Національний технічний університет України
Київський політехнічний інститут ім. Ігоря Сікорського
Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

ЗВІТ
ПРО ПРОХОДЖЕННЯ ПЕРЕДДИПЛОМНОЇ ПРАКТИКИ

Студента(ки) 4 курсу ІП-23 групи
Спеціальності 121 «Інженерія програмного забезпечення»

Зубарева Миколи Костянтиновича

(прізвище, ім'я, по батькові студента)

Термін практики з «20»квітня 2026 р. по «17» травня 2026 р.

База практики: КБІС (Конструкторське бюро інформаційних систем)

(назва підприємства)

Керівник від підприємства

(науковий ступінь, вчене звання, посада)

(прізвище, ім'я, по батькові, підпис)

Керівник практики від
кафедри ІІІ

(науковий ступінь, вчене звання, посада)

(прізвище, ім'я, по батькові, підпис)

Київ-2026 р.

ЗМІСТ

Зміст

1. ПЕРЕДПРОЄКТНЕ ОБСТЕЖЕННЯ ПРЕДМЕТНОЇ ОБЛАСТІ	7
1.1. Постановка завдання дипломного проєктування	7
1.2. Аналіз предметної області	8
1.3. Аналіз існуючих рішень	10
1.3.1. Аналіз відомих програмних продуктів	10
1.3.2. Аналіз відомих алгоритмічних та технічних рішень	15
1.4. Аналіз та моделювання бізнес-процесів	17
1.4.1 Бізнес-процес реєстрації та автентифікації	18
1.4.2 Бізнес-процес створення та публікації опитування	19
1.4.3 Бізнес-процес проходження опитування	20
o Висновки до розділу	21
2. РОЗРОБЛЕННЯ ВИМОГ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	23
2.1. Варіанти використання програмного забезпечення	23
2.2. Розроблення функціональних вимог	38
2.3. Розроблення нефункціональних вимог	41
2.4. Аналіз системних вимог	42
2.5. Аналіз економічних показників програмного забезпечення	44
2.6. Постановка завдання на розробку програмного забезпечення	52
o Висновки до розділу	53
3. КОНСТРУЮВАННЯ ТА РОЗРОБЛЕННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	55
3.1. Архітектура програмного забезпечення	55
3.2. Архітектурні рішення та обґрунтування вибору засобів розробки	59
3.3. Конструювання програмного забезпечення	61
3.3.1. Опис структури бази даних ...	61
3.4. Аналіз безпеки даних	72
o Висновки до розділу	74
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ	76

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

IDE	- Integrated Development Environment - інтегроване середовище розробки.
API	- Application programming interface, прикладний програмний Інтерфейс.
SDK	- Software development kit.
IT	- Інформаційні технології.
ER	- Entity-Relation diagram.
OS	- Операційна система.
БД	- База даних.

ВСТУП

В умовах стрімкої цифровізації усіх сфер суспільного життя збір, обробка та аналіз думок великих аудиторій набувають вирішального значення для прийняття обґрунтованих рішень у бізнесі, освіті, науці та державному управлінні. Онлайн-опитування є одним із найефективніших та найдоступніших інструментів для досягнення цієї мети, оскільки дозволяють охопити широку географічно розподілену аудиторію, автоматизувати збір відповідей та отримувати результати в режимі реального часу.

Актуальність теми дипломного проекту обумовлена зростаючою потребою організацій різних галузей - від бізнесу та HR до наукових установ і державних органів - у зручних, функціональних та безпечних веб-орієнтованих інструментах для створення, поширення та аналізу опитувань. За даними дослідницьких компаній, глобальний ринок програмного забезпечення для опитувань зростає щорічно на 12-15%, а попит на рішення з розширеними аналітичними можливостями значно перевищує пропозицію на ринку відкритого програмного забезпечення.

Провідні тенденції розв'язання задачі автоматизованого збору та аналізу зворотного зв'язку включають: перехід від монолітних до мікросервісних архітектур, широке застосування хмарних технологій та контейнеризації, інтеграцію аналітичних модулів із засобами машинного навчання для виявлення закономірностей у відповідях, а також активне використання SPA-підходу (Single Page Application) для побудови зручного клієнтського інтерфейсу. Сучасний стан розробок характеризується домінуванням комерційних рішень (SurveyMonkey, Google Forms, Typeform), що мають суттєві обмеження: закритий вихідний код, обмежений безкоштовний функціонал та неможливість повного контролю над даними.

У рамках даного дипломного проекту розроблено програмне забезпечення веб-орієнтованої системи створення, проведення та аналізу онлайн-опитувань Survey App. Система реалізована як повностекний веб-

додаток з архітектурою клієнт-сервер, що включає: React SPA з TypeScript на клієнтській стороні, RESTful API на базі Node.js та Express з TypeScript на серверній стороні, реляційну базу даних PostgreSQL з ORM Prisma, а також контейнеризацію через Docker та Nginx.

Призначення системи - надати організаціям та окремим авторам зручний інструмент для самостійного розгортання платформи опитувань з повним контролем над даними. Мета дипломного проєкту - розробити функціонально повну, безпечну та продуктивну веб-орієнтовану систему, що підтримує повний цикл роботи з онлайн-опитуваннями: від створення та публікації до збору відповідей та інтерактивного аналізу результатів.

Можливі сфери застосування розробленої системи охоплюють: маркетингові дослідження (NPS-опитування, збір відгуків про продукти), управління персоналом (опитування задоволеності співробітників, зворотний зв'язок після тренінгів), освіти (тестування знань, оцінювання якості викладання), академічні дослідження (соціологічні та психологічні опитування), а також оцінювання якості державних послуг.

1. ПЕРЕДПРОЄКТНЕ ОБСТЕЖЕННЯ ПРЕДМЕТНОЇ ОБЛАСТІ

1.1. Постановка завдання дипломного проєктування

Дипломне проєктування передбачає виконання наступних завдань:

- аналіз предметної області онлайн-опитувань та опис її ключових бізнес-процесів, визначення загального завдання розробки у рамках дипломного проєкту;
- аналіз існуючих програмних рішень у сфері онлайн-опитувань та порівняльний аналіз їхніх функціональних можливостей;
- аналіз алгоритмічних та технічних рішень для побудови веборієнтованих систем збору та аналізу даних;
- аналіз та моделювання ключових бізнес-процесів системи з використанням нотації BPMN;
- розроблення функціональних, нефункціональних та системних вимог до програмного забезпечення;
- аналіз економічних показників програмного забезпечення дипломного проєкту;
- постановка завдання на розробку програмного забезпечення;
- проєктування архітектури системи: вибір архітектурного стилю, проєктування схеми бази даних та REST API;
- обґрунтування вибору засобів розробки програмного забезпечення та технологічного стеку;
- конструювання та реалізація програмного забезпечення: серверна частина (Node.js, Express, Prisma), клієнтська частина (React, TypeScript), аналітичний модуль;
- аналіз безпеки даних програмного забезпечення;
- аналіз якості та тестування програмного забезпечення;

- розгортання програмного забезпечення з використанням Docker та Nginx;
- створення супроводжувальної документації до розробленого програмного забезпечення.

1.2. Аналіз предметної області

Предметною областю даної роботи є веборієнтовані інформаційні системи для збору, обробки та аналізу структурованих даних у формі опитувань. Онлайн-опитування являють собою цифровий метод збору інформації, при якому респонденти відповідають на запитання через веб-інтерфейс, а відповіді автоматично зберігаються та агрегуються для подальшого аналізу. На відміну від традиційних паперових анкет, веборієнтовані системи опитувань забезпечують автоматизований збір відповідей, мінімізують людський фактор при обробці результатів та надають можливість аналізу даних у реальному часі.

Стрімкий розвиток цієї сфери розпочався у 2000-х роках разом із поширенням широкосмугового Інтернету та масовою веб-доступністю, а новий етап еволюції пов'язаний із впровадженням мобільних технологій, що зробило участь в опитуваннях зручною для будь-якого пристрою. Сучасні системи онлайн-опитувань трансформуються з простих форм збору даних на комплексні платформи дослідницьких даних з розвиненими аналітичними можливостями.

Згідно з дослідженнями у сфері інформаційних систем, ефективність збору зворотного зв'язку безпосередньо залежить від зручності інструменту, різноманітності підтримуваних типів питань та доступності аналітики для авторів опитувань.

Дослідження також показують, що персоналізований дизайн опитувань та умовна логіка переходу між питаннями підвищують рівень завершення опитувань на 20-35% порівняно зі стандартними лінійними формами.

Аналіз предметної області дозволяє виділити ключові категорії користувачів систем онлайн-опитувань та їх специфічні потреби.

Комерційні організації використовують опитування насамперед для вимірювання індексу споживчої лояльності (Net Promoter Score, NPS), дослідження задоволеності клієнтів (CSAT) та збору відгуків про продукти. Для цієї категорії критично важливими є підтримка NPS-типу питань, можливість автоматичного розрахунку показників та інтеграція результатів з CRM-системами через вебхуки.

HR-департаменти та служби управління персоналом активно застосовують опитування для оцінювання задоволеності та залученості співробітників (Employee Engagement Survey), збору зворотного зв'язку після тренінгів та оцінювання корпоративної культури. Специфічною потребою цієї категорії є забезпечення анонімності відповідей та підтримка матричних питань для багатокритеріального оцінювання.

Освітні заклади (університети, школи, навчальні центри) використовують системи опитувань для проведення тестувань, збору відгуків студентів про якість викладання та оцінювання ефективності навчальних програм. Для освітнього сектору важливими є підтримка обмеження часу на проходження, запобігання повторним відповідям та експорт результатів у форматах, сумісних з академічним програмним забезпеченням.

Дослідницькі організації та соціологічні служби застосовують онлайн-опитування для проведення широкомасштабних соціологічних, маркетингових та наукових досліджень. Для цієї категорії першорядне значення мають статистична достовірність зібраних даних, підтримка складних типів питань (матриця, шкала Лікерта) та можливість експорту даних у форматах для статистичного аналізу (CSV, SPSS).

На сучасному етапі розвитку ІТ реалізація систем онлайн-опитувань характеризується використанням клієнт-серверних архітектур із застосуванням SPA-підходу на клієнтській стороні та RESTful або GraphQL

API - на серверній. Бізнес-процес функціонування типової системи онлайн-опитувань передбачає: реєстрацію та автентифікацію авторів, створення опитування з налаштуванням структури та доступу, публікацію та поширення за посилання, збір відповідей респондентів та автоматизовану обробку та візуалізацію результатів.

Аналіз предметної області виявляє суперечність між зростаючими потребами організацій у гнучких та функціональних інструментах для опитувань та обмеженнями існуючих рішень: комерційні продукти є дорогими та не надають повного контролю над даними, тоді як відкриті рішення часто поступаються за зручністю та сучасністю інтерфейсу. Саме це протиріччя визначає актуальність розробки власної відкритої системи на сучасному технологічному стеку.

1.3. Аналіз існуючих рішень

Проаналізуємо відоме на сьогодні алгоритмічне забезпечення у даній області та технічні рішення, що допоможуть у реалізації Survey App. Далі будуть розглянуті готові програмні рішення, допоміжні програмні засоби та засоби розробки.

1.3.1. Аналіз відомих програмних продуктів

У межах предметної області онлайн-опитувань існує значна кількість програмних продуктів. Для аналізу обрано найбільш репрезентативні рішення: Google Forms, SurveyMonkey, Typeform та LimeSurvey - як приклад відкритого рішення.

Google Forms є найпоширенішим безкоштовним інструментом для створення онлайн-опитувань, що входить до екосистеми Google Workspace. Система підтримує базові типи питань (вибір з варіантів, шкала, текст, завантаження файлів), автоматичну агрегацію відповідей у Google Sheets та базову візуалізацію результатів у вигляді кругових та стовпчастих діаграм. Система має реєстрацію користувачів через Google-акаунт та обмежені

можливості кастомізації зовнішнього вигляду. Суттєвими недоліками є відсутність NPS-типу питань, обмежена аналітика, відсутність умовної логіки в безкоштовній версії та повна залежність від інфраструктури Google (рис. 1.1) [1].

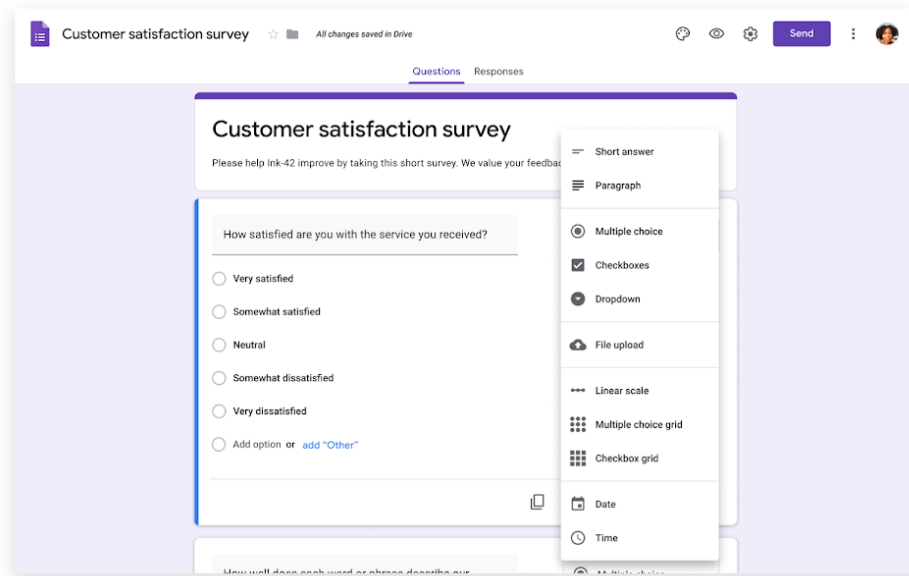


Рисунок 1.1 - Інтерфейс конструктора опитувань Google Forms

SurveyMonkey є одним із провідних комерційних рішень на ринку, що надає розширений функціонал для бізнес-досліджень. Платформа підтримує понад 15 типів питань, включаючи матричні питання та шкалу Лікерта, умовну логіку (skip logic), розширену аналітику з фільтрацією та порівнянням результатів, а також інтеграцію з популярними бізнес-системами (Salesforce, HubSpot, Slack). Серед переваг - якісний UX та широкі можливості налаштування брендингу. Ключовим обмеженням є висока вартість підписки (від 25 до 99 USD на місяць за просунуті функції) та закритий вихідний код (рис. 1.2) [2].

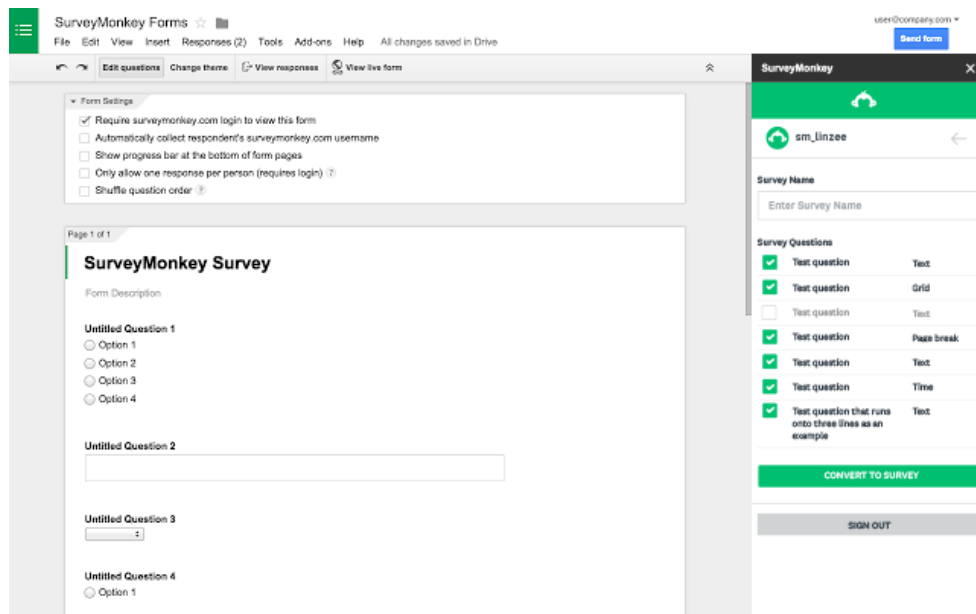


Рисунок 1.2 - Інтерфейс платформи SurveyMonkey

Туреform відрізняється унікальним підходом до UX: опитування відображаються у форматі «одне питання за раз» з анімованими переходами, що забезпечує вищий рівень залученості респондентів. Система підтримує умовну логіку, інтеграції через Zapier та власний API. Недоліком є обмежена кількість типів питань у базовій версії (8 типів) та відсутність матричних питань у безкоштовному плані. Вартість комерційного плану - від 25 USD на місяць (рис. 1.3) [3].

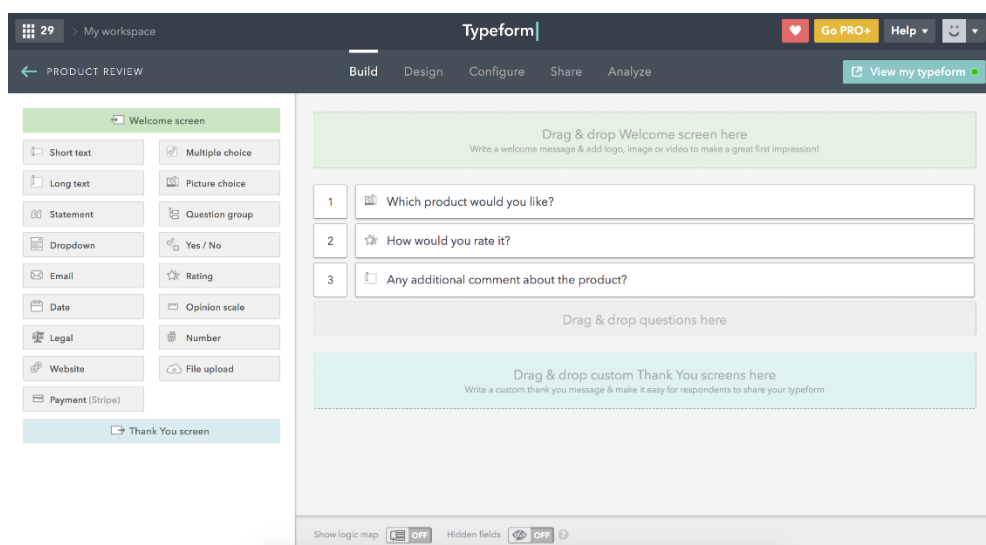


Рисунок 1.3 - Інтерфейс платформи Typeform

LimeSurvey є найбільш функціональним відкритим рішенням у своїй категорії, що підтримує понад 28 типів питань, розширену умовну логіку та

локалізацію на 80+ мов. Система розгортається на власному сервері, що забезпечує повний контроль над даними. Однак LimeSurvey має суттєві недоліки: застарілий інтерфейс, складність налаштування та відсутність сучасного TypeScript-стеку, що ускладнює розширення функціоналу [4].

Для порівняння проєкту з аналогом можна скористатись таблицею 1.3.

Таблиця 1.3 - Порівняння з аналогами

Функціонал	Survey App (дипломний проєкт)	GoogleForms	SurveyMonkey	TypeForm	Пояснення
Безкоштовне використання	-	+	±	±	Survey App - повністю безкоштовні; SM та Typeform обмежують кількість відповідей і питань
Відкритий вихідний код	+	-	-	-	Відкритий код дозволяє власне розгортання та модифікацію без ліцензійних відрахувань
Кількість типів питань	10	9	15	8	Survey App покриває всі практично необхідні типи
Завантаження файлів	+	+	+	+	Підтримується всіма системами

5 рівнів видимості	+	-	±	-	Найбільш гнучка модель серед усіх порівнювани х систем
Аналітика в реальному часі	+	±	+	±	Survey App та SurveyMonke у - розширеною real-time аналітикою серед порівнювани х
Експорт CSV	+	+	+	+	Survey App генерує CSV з BOM для коректного відкриття в Excel з підтримкою кирилиці
Публічний каталог опитувань	+	-	-	-	Унікальна функція: /explore з пошуком, тегами, закладками - аналог магазину опитувань
Інтеграція ІІІ	+	-	±	±	Система через prompt- інжиніринг автоматично генерує та зберігає в БД повноцінну

					структуру чернетки опитування з правильними типами питань за введеною темою.
Публічна сторінка автора	+	-	-	-	/u/{username} - публічна сторінка зі статистикою
Автозбереження прогресу	+	-	+	+	Прогрес зберігається кожні 2 секунди через sessionKey; відновлюється при повторному відкритті

1.3.2. Аналіз відомих алгоритмічних та технічних рішень

При розробці веборієнтованої системи онлайн-опитувань ключовими технічними задачами є організація клієнт-серверної взаємодії, вибір архітектурного стилю інтерфейсу, забезпечення безпеки та реалізація інтелектуального модуля генерації контенту.

1. Порівняльний аналіз серверних платформ

Для реалізації логіки сервера проаналізовано середовища Node.js (Express/Fastify) та Python (FastAPI/Django). Хоча Python має переваги у сфері Data Science, Node.js є більш раціональним вибором для даної системи:

- Isomorphic Development: Використання TypeScript на обох рівнях (Frontend/Backend) дозволяє створювати спільні інтерфейси даних, що мінімізує помилки інтеграції.

- Продуктивність: Асинхронна модель Event Loop у Node.js ідеально підходить для високого I/O навантаження під час масового збору відповідей.
- Екосистема: Наявність зрілих ORM та бібліотек для валідації JSON-схем прискорює розробку модуля обробки результатів.

2. Обґрунтування вибору СУБД та інструментів доступу до даних

Для зберігання структурованих даних опитувань (користувачі, форми, результати) обрано реляційну модель. Порівняння інструментів взаємодії з БД наведено нижче:

1. Raw SQL: Максимальна продуктивність, але висока складність підтримки та ризик SQL-ін'єкцій.
2. Класичні ORM (TypeORM/Sequelize): Високий рівень абстракції, проте складність у типізації складних запитів.
3. Prisma ORM: Обране рішення. Надає типобезпечний клієнт, що автоматично генерується на основі декларативної схеми. Це гарантує цілісність даних на етапі компіляції та спрощує міграції при зміні структури опитувань.

3. Алгоритми автентифікації та безпеки

Розглянуто два основні методи керування сесіями: Stateful (Session-based) та Stateless (JWT).

Для проєкту обрано механізм JWT (JSON Web Tokens) [5]. Це дозволяє реалізувати масштабовану архітектуру, де сервер не зберігає стан сесії в пам'яті, що спрощує роботу в Docker-контейнерах. Для захисту облікових даних використовується алгоритм bcrypt з коефіцієнтом складності 12, що є стійким до brute-force атак завдяки механізму "солі" (salting).

4. Аналіз моделей штучного інтелекту для модуля генерації

Для реалізації функції автоматичного створення опитувань проведено порівняльний аналіз доступних LLM (Large Language Models) (таблиця 1.3.1):

Таблиця 1.3.1 - Аналіз моделей штучного інтелекту

Характеристика	OpenAI GPT-4	Anthropic Claude	Google Gemini 2.5 Flash
Вартість (API)	Висока	Середня	Безкоштовно (в межах лімітів)
Швидкість (Latency)	Середня	Висока	Висока
Форматування JSON	Відмінне	Відмінне	Високе

Для розробки ДП обрано Google Gemini 2

.5 Flash. Модель демонструє оптимальний баланс між швидкістю генерації структурованого JSON та доступністю (безкоштовний ліміт 1 500 запитів/добу). Алгоритмічно взаємодія будується на основі Few-shot prompting, де моделі надається приклад схеми опитування, після чого вона генерує валідну структуру, яка інтегрується в базу даних системи без додаткової ручної обробки.

1.4. Аналіз та моделювання бізнес-процесів

Для побудови програмного забезпечення визначено та змодельовано ключові бізнес-процеси системи онлайн-опитувань. Моделювання виконано з використанням нотації BPMN (Business Process Model and Notation), що є міжнародним стандартом для графічного опису бізнес-процесів [6,7]. Розглянуто три ключові бізнес-процеси: реєстрація та аутентифікація користувача, створення та публікація опитування, а також проходження опитування та збір відповідей.

1.4.1 Бізнес-процес реєстрації та автентифікації

Бізнес-процес реєстрації та автентифікації користувача охоплює дії від першого відкриття системи до успішного входу в особистий кабінет (рис. 1.4).

Опис моделі бізнес-процесу реєстрації:

1. Користувач переходить на сторінку реєстрації (/register);
2. Користувач заповнює поля реєстраційної форми: ім'я, email та пароль (мін. 8 символів);
3. Система виконує клієнтську валідацію форми: якщо поля не відповідають вимогам, відображаються повідомлення про помилки без відправки запиту;
4. Після успішної клієнтської валідації система надсилає POST-запит до /api/auth/register;
5. Сервер перевіряє унікальність email в базі даних: якщо email вже зареєстрований - повертає помилку 409;
6. При унікальному email сервер хешує пароль алгоритмом bcrypt (12 раундів), зберігає користувача та повертає JWT-токен;
7. Токен зберігається у localStorage клієнта, користувач перенаправляється на /dashboard.

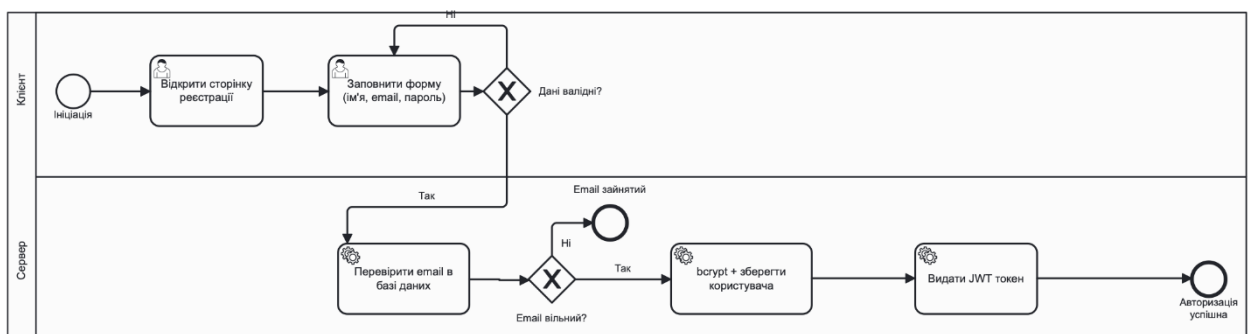


Рисунок 1.4 - BPMN-модель бізнес-процесу реєстрації користувача

1.4.2 Бізнес-процес створення та публікації опитування

Бізнес-процес створення та публікації опитування охоплює повний цикл роботи автора від ініціювання нового опитування до його доступності для респондентів (рис. 1.5).

Опис моделі бізнес-процесу:

1. Авторизований користувач з роллю Creator або Admin переходить до конструктора опитувань (/surveys/new);
2. Автор заповнює загальні налаштування: назву, опис, дати початку та кінця, параметри анонімності;
3. Автор додає питання, вибираючи тип кожного питання з 10 доступних типів та налаштовуючи параметри;
4. Для кожного питання автор може налаштувати умовну логіку (skip logic): умову спрацювання та цільове питання;
5. Система виконує валідацію: перевіряє наявність назви та хоча б одного питання з непустим текстом;
6. При успішній валідації автор може зберегти опитування як чернетку (статус DRAFT) або одразу опублікувати (статус ACTIVE);
7. Автор налаштовує рівень доступу: публічне, тільки за посиланням, з паролем або для списку email.

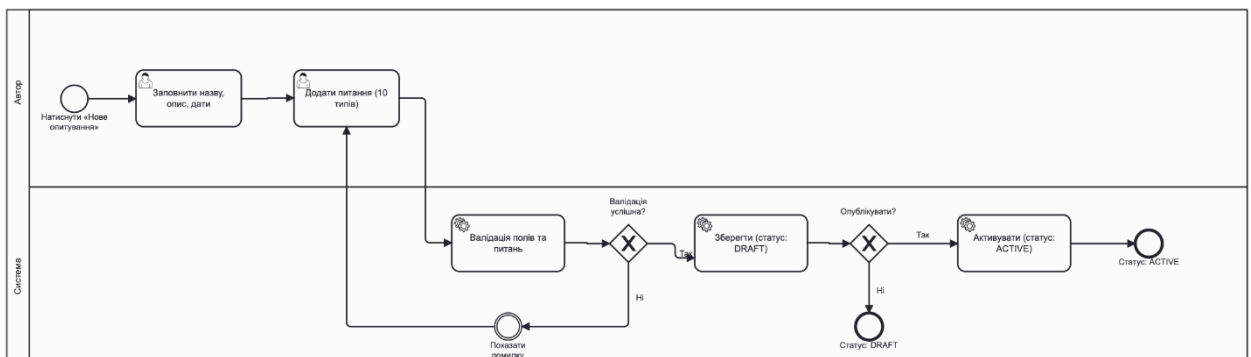


Рисунок 1.5 - BPMN-модель бізнес-процесу створення та публікації опитування

1.4.3 Бізнес-процес проходження опитування

Бізнес-процес проходження опитування охоплює дії респондента від переходу за посиланням до успішного відправлення відповідей (рис. 1.6).

Опис моделі бізнес-процесу:

1. Респондент переходить за посиланням виду /s/{survey-id};
2. Система перевіряє статус опитування: якщо статус не ACTIVE або порушено часові обмеження - відображається відповідне повідомлення;
3. Якщо опитування захищено паролем (visibility = PASSWORD), відображається форма введення пароля; при невірному паролі доступ заблоковано;
4. Система завантажує збережений прогрес (якщо є) з бази даних за ключем сесії;
5. Респондент послідовно відповідає на питання; після кожної відповіді клієнт виконує оцінку умовної логіки - визначає наступне питання;
6. Прогрес автоматично зберігається кожні 2 секунди через API (/api/logic/surveys/{id}/progress);
7. Після відповіді на останнє питання респондент підтверджує відправлення форми;
8. Система перевіряє заповненість всіх обов'язкових питань та обмеження на повторні відповіді;
9. При успішній валідації відповіді зберігаються в базі даних, відображається сторінка подяки.

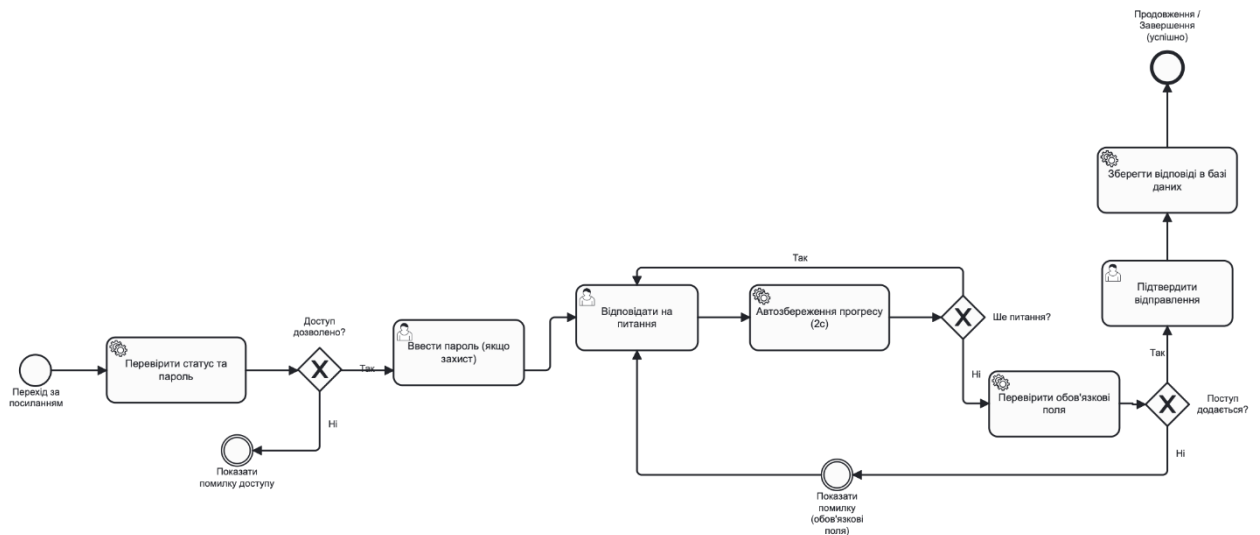


Рисунок 1.6 - BPMN-модель бізнес-процесу проходження опитування

о Висновки до розділу

У першому розділі виконано передпроектне обстеження предметної області веборієнтованих систем онлайн-опитувань.

Визначено повний перелік завдань дипломного проектування з 14 пунктів, що охоплюють повний цикл розробки програмного забезпечення - від аналізу предметної області до створення документації.

Проведено аналіз предметної області, що дозволив виявити основні категорії користувачів систем онлайн-опитувань (бізнес, HR, освіта, дослідники), їх специфічні потреби та ключові функціональні блоки сучасної системи опитувань. Встановлено суперечність між зростаючими потребами організацій у гнучких інструментах та обмеженнями існуючих комерційних рішень.

Виконано порівняльний аналіз трьох провідних систем: Google Forms, SurveyMonkey та Typeform за 15 критеріями (таблиця 1.1). Показано, що розроблена система Survey App перевершує всі аналоги за ключовими критеріями та пропонує унікальні можливості, відсутні у конкурентів: публічний каталог опитувань з пошуком та тегами, інтеграцію AI-генерації через Google Gemini API, теплову карту активності та систему 5 рівнів видимості.

Проаналізовано технічні рішення, покладені в основу розробки. Обґрунтовано вибір: SPA-архітектура (React + TypeScript) для клієнтської частини, Node.js + Express для серверної, Prisma ORM для типобезпечного доступу до PostgreSQL, JWT + bcrypt для автентифікації та Google Gemini API для AI-генерації опитувань. Для кожного технічного рішення виконано порівняльний аналіз з альтернативами та надано обґрунтування вибору.

Змодельовано три ключові бізнес-процеси системи з використанням нотації BPMN: реєстрацію користувача, створення та публікацію опитування (включаючи AI-асистований режим через Gemini), проходження опитування та збір відповідей. Для кожного процесу описано основний та альтернативні потоки виконання.

2. РОЗРОБЛЕННЯ ВИМОГ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

2.1. Варіанти використання програмного забезпечення

Основна задача програмного забезпечення полягає у створенні зручної, безпечної та функціонально повної веборієнтованої платформи для створення, проведення та аналізу онлайн-опитувань [8]. Система поєднує зручний конструктор опитувань з підтримкою десяти типів питань, гнучку систему управління доступом, розширений аналітичний модуль та інструменти командної співпраці.

До головного функціоналу системи належать: реєстрація та авторизація користувачів з підтримкою трьох ролей (Respondent, Creator, Admin), управління профілем з публічною сторінкою, перегляд публічного каталогу опитувань з пошуком та тегами, проходження опитувань з підтримкою умовної логіки та автозбереженням прогресу. Для авторів опитувань доступні: конструктор з десятима типами питань, публікація з п'ятьма рівнями доступу, розширена аналітика результатів, керування командою. Повний перелік функцій представлено на рисунку 2.1 у вигляді діаграми варіантів використання.

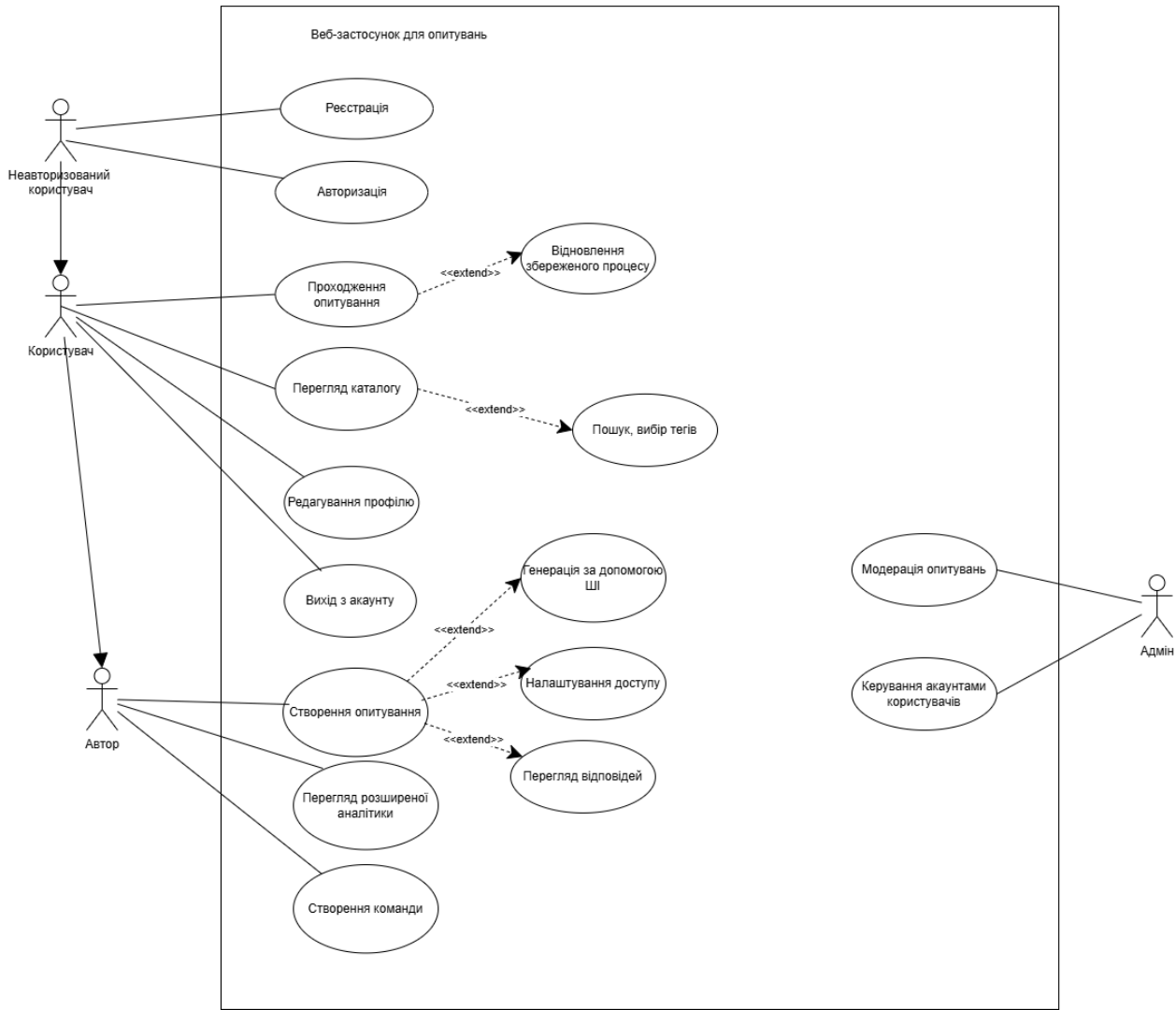


Рисунок 2.1 - Діаграма варіантів використання

В таблицях 2.1 - 2.16 наведено опис варіантів використання програмного забезпечення.

Таблиця 2.1 - Варіант використання UC-01

Use case name	Реєстрація користувача
Use case ID	UC-01
Goals	Реєстрація нового користувача в системі
Actors	Неавторизований користувач (гість)
Trigger	Користувач бажає зареєструватись у системі
Pre-conditions	-

Flow of Events	Користувач переходить на сторінку реєстрації (/register). У поля для реєстрації вводяться відповідні дані: ім'я, електронна пошта та пароль (мінімум 8 символів). Після заповнення форми користувач натискає кнопку реєстрації. Система виконує клієнтську валідацію. Користувач перенаправляється на головну сторінку.
Extension	У випадку введення некоректних даних кнопка реєстрації стає неактивною. Якщо конкретне поле введено некоректно - воно підсвічується червоним написом з поясненням.
Post-Condition	Створення облікового запису користувача в базі даних; перенаправлення на головну

Таблиця 2.2 - Варіант використання UC-02

Use case name	Авторизація користувача
Use case ID	UC-02
Goals	Вхід зареєстрованого користувача до системи
Actors	Неавторизований користувач (гість)
Trigger	Користувач бажає увійти до свого акаунту
Pre-conditions	Користувач зареєстрований у системі
Flow of Events	Користувач переходить на сторінку входу. У відповідні поля вводяться електронна пошта та пароль. Після натискання кнопки «Увійти» система перевіряє введені дані; при успішній перевірці - авторизує користувача у системі. Користувач перенаправляється до особистого кабінету.
Extension	При введенні невірної електронної пошти або пароля система відображає повідомлення «Невірний email або

	пароль» без уточнення, яке саме поле хибне (захист від перебору). Після 5 невдалих спроб можливість входу для даного користувача тимчасово блокується на 15 хвилин (захист від спаму).
Post-Condition	Користувач успішно авторизований; йому надано доступ до функцій особистого кабінету.

Таблиця 2.3 - Варіант використання UC-03

Use case name	Проходження опитування
Use case ID	UC-03
Goals	Заповнення та відправлення відповідей на активне опитування
Actors	Користувач
Trigger	Користувач отримав посилання на опитування або знайшов його у каталозі
Pre-conditions	Опитування є активним; поточний час відповідає встановленим часовим рамкам проведення
Flow of Events	Користувач відкриває сторінку опитування. Якщо воно захищено паролем - відображається форма введення пароля. Система завантажує збережений прогрес (якщо такий є). Користувач послідовно відповідає на питання; після кожної відповіді система перевіряє умовну логіку (щоб визначити, яке питання показати наступним). Прогрес автоматично зберігається. Після останнього питання користувач підтверджує відправку. Система перевіряє заповнення обов'язкових полів та зберігає відповіді.

Extension	Якщо опитування неактивне або завершився час його проведення - відображається повідомлення про недоступність. При введенні невірної пароля доступ блокується. Якщо не заповнені обов'язкові поля - виводиться повідомлення про помилку, а відправка скасовується. Якщо автор заборонив повторне проходження, а користувач вже відповідав раніше - система повідомляє, що відповідь вже зараховано.
Post-Condition	Відповіді збережено

Таблиця 2.4 - Варіант використання UC-04

Use case name	Відновлення збереженого процесу проходження
Use case ID	UC-04
Goals	Продовження раніше розпочатого, але не завершеного опитування
Actors	Користувач
Trigger	Користувач раніше розпочав проходження і закрив сторінку
Pre-conditions	У системі або пам'яті браузера існує збережений прогрес для цього опитування
Flow of Events	Користувач повторно переходить за посиланням на опитування. Система автоматично розпізнає наявність попередньої сесії та знаходить збережений прогрес. Відновлюються попередньо введені відповіді та поточний крок. Система відображає форму у тому вигляді, на якому зупинився користувач. Користувач продовжує проходження.

Extension	Якщо опитування було суттєво змінено автором (наприклад, додано нові питання) після того, як користувач зберіг прогрес, система відображає попередження та пропонує почати спочатку або спробувати продовжити.
Post-Condition	Прогрес відновлено; користувач продовжує проходження з місця зупинки

Таблиця 2.5 - Варіант використання UC-05

Use case name	Редагування профілю
Use case ID	UC-05
Goals	Оновлення персональних даних та налаштувань акаунту користувача
Actors	Користувач, Автор, Адмін
Trigger	Користувач бажає змінити дані свого профілю
Pre-conditions	Користувач авторизований у системі
Flow of Events	Користувач переходить до розділу налаштувань профілю. Змінює доступні поля: ім'я, унікальний псевдонім (для персональної публічної сторінки), коротку біографію, локацію, вебсайт. Завантажує нове фото профілю через форму. Натискає «Зберегти зміни».
Extension	Якщо введений унікальний псевдонім вже зайнятий іншим користувачем, система видає попередження. Якщо завантажене фото перевищує допустимий розмір або має невідпущуваний формат - відображається помилка завантаження.
Post-Condition	Дані профілю оновлено

Таблиця 2.6 - Варіант використання UC-06

Use case name	Перегляд каталогу опитувань
Use case ID	UC-06
Goals	Перегляд та пошук публічних опитувань у загальному каталозі
Actors	Користувач, Автор
Trigger	Користувач бажає знайти публічне опитування
Pre-conditions	-
Flow of Events	Користувач відкриває сторінку загального каталогу. Система відображає список активних публічних опитувань, розділений на сторінки. Користувач може скористатися пошуком за ключовим словом, відфільтрувати список за категоріями чи застосувати сортування (за датою публікації, популярністю, кількістю учасників). Для переходу до проходження або перегляду деталей користувач натискає на картку обраного опитування.
Extension	Якщо результати пошуку порожні - система відображає повідомлення «Нічого не знайдено» з пропозицією змінити фільтри.
Post-Condition	Список опитувань відображено відповідно до застосованих фільтрів

Таблиця 2.7 - Варіант використання UC-07

Use case name	Пошук та вибір тегів
Use case ID	UC-07

Goals	Фільтрація каталогу опитувань за тематичними тегами
Actors	Користувач, Автор
Trigger	Користувач переглядає каталог і хоче знайти опитування певної тематики
Pre-conditions	Система містить хоча б один публічний тег
Flow of Events	Система відображає список доступних тематичних категорій (наприклад: Маркетинг, HR, Освіта, Дослідження тощо). Користувач обирає одну або кілька категорій. Система миттєво оновлює список опитувань, залишаючи лише ті, що відповідають обраним темам. Повторне натискання на обрану категорію скасовує цей фільтр.
Extension	Якщо немає жодного активного опитування, що відповідає обраній комбінації тем, система сповіщає про відсутність результатів.
Post-Condition	Каталог відфільтровано за обраними тегами

Таблиця 2.8 - Варіант використання UC-08

Use case name	Вихід з акаунту
Use case ID	UC-08
Goals	Завершення сесії авторизованого користувача
Actors	Користувач, Автор, Адмін
Trigger	Користувач бажає вийти з системи
Pre-conditions	Користувач авторизований у системі
Flow of Events	Користувач натискає кнопку «Вийти» в меню навігації. Система безпечно завершує активний сеанс та очищає

	локальні дані авторизації. Після цього користувач автоматично перенаправляється на головну сторінку входу.
Extension	-
Post-Condition	Сесію успішно завершено; доступ до особистого кабінету закрито до наступної авторизації.

Таблиця 2.9 - Варіант використання UC-09

Use case name	Створення опитування
Use case ID	UC-09
Goals	Розробка та збереження нового опитування у системі
Actors	Автор, Адмін
Trigger	Автор бажає створити нове опитування
Pre-conditions	Користувач має відповідні права (Автор або Адміністратор)
Flow of Events	Автор переходить до розділу створення опитувань або обирає готовий шаблон. За потреби використовує інструмент штучного інтелекту: вводить тему, і система автоматично формує структуру питань. Автор заповнює або редагує назву, опис, терміни проведення та налаштування анонімності. Додає необхідну кількість питань, обираючи їх типи з доступного переліку (один варіант, кілька варіантів, текст, рейтинг, шкала тощо). По завершенню зберігає проєкт як чернетку або відразу публікує його.
Extension	Якщо автор не вказав назву опитування, збереження блокується. Якщо в опитуванні немає жодного питання, виводиться попередження. При збої в роботі штучного

	інтелекту система пропонує повторити спробу або продовжити створення вручну.
Post-Condition	Нове опитування успішно збережено у базі даних як чернетка або активне опитування.

Таблиця 2.10 - Варіант використання UC-10

Use case name	Генерація опитування за допомогою ІІІ
Use case ID	UC-10
Goals	Автоматичне створення структури та питань опитування на основі заданої теми
Actors	Автор, Адмін
Trigger	Автор у конструкторі опитувань обирає функцію генерації за допомогою ІІІ
Pre-conditions	Автор авторизований; інтеграція з сервісом ІІІ налаштована та активна
Flow of Events	Автор вводить бажану тему (наприклад, «Оцінка задоволеності персоналу»). Натискає кнопку «Генерувати». Система звертається до сервісу штучного інтелекту, передаючи задані параметри. Сервіс повертає структурований перелік питань із відповідними типами та варіантами відповідей. Конструктор опитувань автоматично заповнюється отриманими даними. Автор переглядає результат та за потреби вносить власні корективи.
Extension	Якщо сервіс ІІІ тимчасово недоступний або час очікування вичерпано, система сповіщає про помилку. Якщо введена тема порушує правила безпеки сервісу ІІІ, відображається

	повідомлення про блокування запиту з пропозицією змінити тему.
Post-Condition	Форму конструктора успішно заповнено згенерованими питаннями; автор готовий до подальшого редагування або публікації.

Таблиця 2.11 - Варіант використання UC-11

Use case name	Налаштування доступу до опитування
Use case ID	UC-11
Goals	Визначення рівня видимості та умов доступу до опитування
Actors	Автор, Адмін
Trigger	Автор бажає налаштувати хто може проходити опитування
Pre-conditions	Опитування вже створено у системі
Flow of Events	Автор переходить до налаштувань конкретного опитування. Обирає рівень доступу: публічний (відображається в загальному каталозі), за прямим посиланням, тільки для зареєстрованих користувачів, за паролем або за списком дозволених електронних адрес. Якщо обрано захист паролем - автор задає пароль. Якщо за списком адрес - вводить перелік дозволених учасників. За потреби встановлює обмеження на максимальну кількість відповідей. Зберігає зміни..
Extension	Якщо встановлений пароль є занадто коротким (менше встановленого ліміту), система просить придумати складніший пароль.

Post-Condition	Налаштування конфіденційності успішно застосовані; доступ до опитування регулюється відповідно до обраних правил.
----------------	---

Таблиця 2.12 - Варіант використання UC-12

Use case name	Перегляд розширеної аналітики
Use case ID	UC-12
Goals	Детальний аналіз результатів опитування з інтерактивними графіками
Actors	Автор, Адмін
Trigger	Автор бажає проаналізувати результати опитування
Pre-conditions	Опитування містить хоча б одну відповідь
Flow of Events	Автор відкриває сторінку аналітики обраного опитування. Система підраховує дані та відображає загальні показники: кількість відповідей, відсоток завершених проходжень, середній час заповнення. У розділі результатів відображаються графіки та діаграми для кожного питання окремо. Додатково доступні вкладки з картою активності по днях та годинах, воронкою відмов (на якому етапі учасники найчастіше припиняли заповнення) та статистикою використаних пристроїв. За необхідності автор завантажує результати у вигляді табличного звіту.
Extension	Якщо для побудови складних графіків (наприклад, воронки) зібрано недостатньо даних, система виводить інформаційне повідомлення про це.

Post-Condition	Аналітичні дані успішно відображено; користувач може зберегти звіт на свій пристрій.
----------------	--

Таблиця 2.13 - Варіант використання UC-13

Use case name	Перегляд відповідей
Use case ID	UC-13
Goals	Перегляд індивідуальних відповідей респондентів на опитування
Actors	Автор, Адмін
Trigger	Автор бажає переглянути відповіді конкретних респондентів
Pre-conditions	Опитування містить хоча б одну відповідь
Flow of Events	Автор переходить до розділу списку відповідей. Система відображає перелік усіх учасників, які завершили опитування. При виборі конкретної анкети показується дата проходження, інформація про учасника, витрачений час та перелік наданих відповідей на кожне запитання. Автор має можливість видалити окрему відповідь, якщо вона вважається недійсною.
Extension	Якщо опитування анонімне - дані респондента не відображаються.
Post-Condition	Список відповідей відображено.

Таблиця 2.14 - Варіант використання UC-14

Use case name	Створення команди
Use case ID	UC-14

Goals	Створення спільного робочого простору для колективного управління опитуваннями
Actors	Автор, Адмін
Trigger	Автор хоче залучити колег до спільної роботи над опитуваннями
Pre-conditions	Користувач має відповідний рівень доступу для створення команд
Flow of Events	Автор відкриває розділ управління командами і натискає кнопку створення нової. Вводить назву команди. Система формує робочий простір і призначає творця його власником. Далі автор запрошує інших учасників за їхніми електронними адресами, призначаючи їм рівні доступу (редактор або просто глядач). Запрошені користувачі отримують відповідне сповіщення. Автор переносить потрібні опитування до спільного простору.
Extension	Якщо введена електронна адреса не зареєстрована в системі, виводиться повідомлення про неможливість запросити користувача. Якщо особа вже є учасником цієї команди, система про це попереджає.
Post-Condition	Команду та робочий простір успішно створено; запрошення учасникам надіслано

Таблиця 2.15 - Варіант використання UC-15

Use case name	Модерація опитувань
Use case ID	UC-15

Goals	Контроль за контентом та управління всіма опитуваннями платформи
Actors	Адмін
Trigger	Адміністратор бажає перевірити або заблокувати опитування
Pre-conditions	Користувач авторизований як Адміністратор
Flow of Events	Адміністратор відкриває панель управління, де відображається загальний список усіх опитувань у системі, незалежно від їх авторів. Переглядаючи список, адміністратор може примусово змінити статус будь-якого опитування (наприклад, закрити його для проходження) або повністю видалити у разі порушення правил платформи.
Extension	-
Post-Condition	Статус обраних опитувань успішно змінено або їх видалено з системи.

Таблиця 2.16 - Варіант використання UC-16

Use case name	Керування акаунтами користувачів
Use case ID	UC-16
Goals	Адміністрування облікових записів та розподіл ролей у системі
Actors	Адмін
Trigger	Адміністратор має надати користувачеві додаткові права або заблокувати його
Pre-conditions	Користувач авторизований як Адміністратор

Flow of Events	Адміністратор відкриває реєстр усіх зареєстрованих користувачів платформи. Має можливість переглянути базову інформацію: ім'я, контактні дані, поточну роль та статус активності. Адміністратор може змінити роль обраного користувача (наприклад, надати права на створення опитувань) або деактивувати обліковий запис (заблокувати доступ до платформи).
Extension	Система блокує можливість випадкового зняття прав Адміністратора з самого себе задля уникнення втрати контролю над системою.
Post-Condition	Налаштування облікових записів та їхні ролі успішно оновлені в системі.

2.2. Розроблення функціональних вимог

Програмне забезпечення використовує монолітну архітектуру з чітким розподілом відповідальності між контролерами. Функціональні вимоги згруповані за модулями системи. В таблиці 2.17 наведено загальну модель вимог.

Таблиця 2.15 - Загальна модель вимог

№	Назва	ID Вимоги	Пріоритети	Ризики
1	Автентифікація	FR-01	Критичний	Високий
2	Управління персональним профілем	FR-02	Високий	Низький
3	Каталог публічних опитувань	FR-03	Високий	Середній
4	Проходження опитування	FR-04	Критичний	Високий
5	Конструктор опитувань	FR-05	Критичний	Високий

6	Інтеграція ІІІ для автоматизації (AI-генерація	FR-06	Середній	Високий
7	Управління життєвим циклом та публікацією	FR-07	Критичний	Середній
8	Налаштування приватності та рівнем доступу	FR-08	Високий	Середній
9	Розширена аналітика	FR-09	Середній	Середній
10	Управління шаблонами	FR-10	Середній	Низький
11	Команди	FR-11	Середній	Середній
12	Адміністрування опитувань	FR-12	Критичний	Високий
13	Адміністрування користувачів	FR-13	Критичний	Високий

Таблиця 2.16 - Перелік функціональних вимог

Назва	Опис
FR-1	Автентифікація та безпека. Неавторизований користувач може зареєструватися та увійти до системи за допомогою електронної пошти та пароля. Система забезпечує хешування паролів (bcrypt), розподіл на три ролі (Admin, Creator, Respondent).
FR-2	Управління персональним профілем. Авторизований користувач має можливість переглядати та редагувати свої особисті дані (username, біографія, локація, вебсайт), завантажувати аватар, а також змінювати глобальні налаштування.
FR-3	Каталог публічних опитувань. Користувачі можуть переглядати загальну сторінку-каталог з усіма відкритими опитуваннями. Реалізовано глобальний пошук, фільтрацію за тематичними тегами.
FR-4	Проходження опитування. Респондент може зручно взаємодіяти з анкетною у покроковому режимі. Підтримується умовна логіка для переходу до певних питань залежно від попередньої відповіді, а також автозбереження прогресу кожні 2 секунди.

FR-5	Конструктор опитувань. Автор має змогу створювати нові анкети за допомогою візуального редактора, використовуючи 10 різних типів питань та налаштовувати їх параметри.
FR-6	Інтеграція ІІІ для автоматизації (AI-генерація). Користувач може ввести бажану тему, а система автоматично згенерує повноцінну структуру опитування з відповідними питаннями та варіантами відповідей за допомогою інтеграції з Google Gemini API.
FR-7	Управління життєвим циклом та публікацією. Автор може змінювати статуси своїх опитувань (чернетка, активне, на паузі, закрите).
FR-8	Налаштування приватності та рівня доступу. Автор може гнучко керувати доступом до опитування, обираючи з 5 рівнів видимості: публічне, за прямим посиланням, приватне, за паролем або за конкретним списком email-адрес. Є змога встановлювати ліміти на кількість відповідей.
FR-9	Розширена аналітика. Система автоматично агрегує зібрані дані та надає їх у вигляді графіків і діаграм. Включає розрахунок NPS Score, матричну теплову карту активності (дні тижня та години), воронку відмов респондентів, аналіз пристроїв та можливість експорту результатів у CSV-файл.
FR-10	Управління шаблонами. Користувачі мають змогу переглядати галерею готових системних шаблонів для швидкого старту роботи.
FR-11	Команди. Реалізовано можливість створення командних робочих просторів для спільної розробки опитувань. Власник може надсилати запрошення на email іншим користувачам та призначати їм відповідні ролі.
FR-12	Адміністрування опитувань. Користувач із правами адміністратора має доступ до глобального списку всіх існуючих у системі опитувань. Він може виконувати модерацію контенту: переглядати, змінювати статус або видаляти будь-яке опитування.

FR-13	Адміністрування користувачів. Адміністратор може переглядати реєстр усіх зареєстрованих акаунтів, контролювати їх статус активності, змінювати системні ролі (наприклад, видавати права Creator) або повністю деактивувати облікові записи порушників.
-------	--

	FR-1	FR-2	FR-3	FR-4	FR-5	FR-6	FR-7	FR-8	FR-9	FR-10	FR-11	FR-12	FR-13
UC-1	+												
UC-2	+												
UC-3				+									
UC-4				+									
UC-5		+											
UC-6			+										
UC-7			+										
UC-8	+												
UC-9					+		+			+			
UC-10						+							
UC-11								+					
UC-12									+				
UC-13													
UC-14											+		
UC-15												+	
UC-16													+

Рисунок 2.3 - Матриця трасування вимог

2.3. Розроблення нефункціональних вимог

Нефункціональні вимоги визначають характеристики та обмеження, яким повинна відповідати система Survey App для забезпечення стабільної та якісної роботи веборієнтованого застосунку. Ці вимоги охоплюють продуктивність, надійність, безпеку, масштабованість та зручність використання.

Вимоги до продуктивності. Час відповіді API для 95% запитів не повинен перевищувати 200 мс при навантаженні 100 одночасних користувачів. Час завантаження основних сторінок за швидкості з'єднання 50 Мбіт/с - не більше 3 секунд. Запити до Google Gemini API обробляються асинхронно з відображенням індикатора очікування; максимальний час очікування відповіді Gemini - 15 секунд, після чого система відображає помилку. Пропускна здатність API - щонайменше 500 запитів/сек.

Вимоги до надійності та доступності. Доступність системи - не нижче 99.5% за місяць. Відновлення після збою - протягом 15 хвилин завдяки Docker-контейнеризації та автоматичному перезапуску (restart: unless-stopped). Дані

зберігаються без втрат завдяки транзакційності PostgreSQL. При тимчасовій недоступності Gemini API - система продовжує роботу без AI-функції (graceful degradation).

Вимоги до безпеки. Передача даних виключно через HTTPS. Паролі - bcrypt-хеші (складність 12). Захист від атак: Helmet.js (HTTP-заголовки), express-rate-limit (200 запитів/15 хв для загальних ендпоінтів, 30 запитів/год для AI-ендпоінтів), валідація через Zod-схеми, параметризовані SQL-запити Prisma. GEMINI_API_KEY зберігається виключно у серверному .env-файлі та ніколи не передається на клієнт.

Вимоги до масштабованості. Stateless JWT-автентифікація (без серверних сесій) дозволяє горизонтальне масштабування. Docker Compose дозволяє незалежне масштабування контейнерів. Redis використовується для кешування, що знижує навантаження на PostgreSQL.

2.4. Аналіз системних вимог

Системні вимоги описують мінімальне програмно-апаратне середовище, у якому веборієнтована система створення, проведення та аналізу онлайн-опитувань може бути успішно зібрана, розгорнута та стабільно експлуатована, а також обмеження, що впливають із обраного технологічного стеку. Для серверної частини додатку на базі платформи Node.js необхідне стабільне середовище виконання сумісної версії, під керуванням якого забезпечується робочий процес обробки бізнес-логіки та запитів до бази даних. На етапі розгортання та запуску у системі мають бути обов'язково налаштовані змінні середовища для збереження секретних ключів автентифікації користувачів, а також параметрів доступу до зовнішнього сервісу штучного інтелекту Google Gemini, який відповідає за автоматизовану генерацію структури анкет за заданою темою. Без коректного підключення до реляційного сховища та попереднього застосування міграцій об'єктно-реляційного відображення ORM Prisma, система не зможе обслуговувати

динамічні компоненти конструктора опитувань, каталог публічних анкет та модулі збору відповідей.

Вимоги до обчислювальних ресурсів і дискового сховища сервера залежать від інтенсивності збору відповідей та загальної кількості одночасних сесій респондентів. Для забезпечення стабільної роботи платформи онлайн-опитувань необхідна конфігурація серверного обладнання з багатоядерним центральним процесором та обсягом оперативної пам'яті, достатнім для паралельної обробки клієнтських запитів, виконання аналітичних обчислень та взаємодії із СУБД. Дисковий простір сервера повинен враховувати обсяг збереження результатів анкетування та окремо передбачати сховище для мультимедійних файлів, які завантажуються респондентами як відповіді на питання відповідного типу. База даних у промисловому продакшн-середовищі орієнтована на реляційну СУБД PostgreSQL, яка забезпечує підтримку транзакційності та цілісності даних; додатково у системі передбачено використання швидкого сховища даних в оперативній пам'яті на базі Redis для оптимізації швидкодії, кешування результатів аналітики та реалізації механізмів обмеження частоти запитів з метою захисту від спаму.

З боку клієнтських робочих місць автори опитувань та респонденти взаємодіють із платформою через сучасні веб-браузери з обов'язково увімкненим виконанням сценаріїв JavaScript, підтримкою захищеного TLS-з'єднання та стабільною швидкістю мережевого підключення. Мережевий канал клієнта є критично важливим для забезпечення коректної роботи підсистеми фонові синхронізації та автоматичного збереження проміжного прогресу проходження анкети, що відбувається кожні дві секунди. Інтерфейс користувача, побудований за архітектурою Single Page Application (SPA), вимагає адаптивної верстки для повного збереження ергономіки та функціональності конструктора й форми проходження на мобільних пристроях і планшетах. Для користувачів з роллю автора, які працюють із великими масивами аналітичних даних (зокрема тепловими картами

активності респондентів та воронками відмов) або завантажують файли, доцільно забезпечити вищу пропускну здатність каналу.

Мережеві та інтеграційні системні вимоги включають безпечний доступ із сервера додатку до сховища даних за захищеним протоколом, дозволений вихідний HTTPS-трафік до API штучного інтелекту Google Gemini, а також належне налаштування сервісу зворотного проксі-сервера та маршрутизації на базі Nginx, який відповідає за SSL-термінацію публічного домену та роздачу статичних файлів клієнтської частини. Для автоматизації процесів збірки й супроводу системи в межах архітектури DevOps використовуються інструменти контейнеризації на базі Docker та Docker Compose. Це унеможливорює виникнення помилок сумісності залежностей та забезпечує автоматичне виконання процесів інсталяції пакетів, генерації клієнтських шарів ORM Prisma та фінальної збірки статичних файлів інтерфейсу перед запуском веб-платформи у промислову експлуатацію.

2.5. Аналіз економічних показників програмного забезпечення

Для планування термінів та визначення фінансових витрат на проектування веборієнтованої системи онлайн-опитувань було використано сучасний алгоритмічний підхід нормування праці - метод пунктів варіантів використання (Use Case Points, UCP). Цей підхід є найбільш об'єктивним для архітектури сучасних вебдодатків, оскільки він дозволяє безпосередньо конвертувати високорівневі функціональні вимоги, зафіксовані у специфікації прецедентів розробки, у кількісні оцінки часових та грошових ресурсів.

Обчислювальна схема методу базується на визначенні початкового нескоригованого показника пункту прецедентів (Unadjusted Use Case Points, UUCP), який формується як лінійна комбінація двох незалежних параметрів: сумарної складності дійових осіб (Unadjusted Actor Weight, UAW) та сумарної складності сценаріїв поведінки системи (Unadjusted Use Case Weight, UUCW). Базова математична модель розрахунку нескоригованого обсягу описується виразом:

$$UUCP = UAW + UUCW$$

Отримане базове значення піддається коригуванню за допомогою двох інтегральних коефіцієнтів, що враховують специфіку обраного технологічного стеку та організаційні параметри процесу розробки:

Коефіцієнт технічної складності (Technical Complexity Factor, TCF), що обчислюється на основі тринадцяти стандартних системних архітектурних вимог (таких як розподіленість, вимоги до безпеки, паралелізм):

$$TCF = 0.6 + (0.01 \times \sum_{i=1}^{13} T_i \times W_i)$$

де T_i - суб'єктивна оцінка ступеня впливу архітектурної вимоги (від 0 до 5), а W_i - нормативна вага чинника.

Коефіцієнт середовища (Environmental Factor, EF), який відображає рівень кваліфікації команди розробників, стабільність вимог та ступінь володіння інструментами автоматизації:

$$EF = 1.4 + (-0.03 \times \sum_{j=1}^{13} E_j \times W_j)$$

де E_j - експертна оцінка фактора команди (від 0 до 5), а W_j - фіксований ваговий коефіцієнт.

Підсумковий скоригований обсяг програмного продукту (Adjusted Use Case Points, AUCP) визначається як добуток початкової оцінки на знайдені коефіцієнти складності:

$$AUCP = UUCP \times TCF \times EF$$

Фінальний розрахунок сумарної трудомісткості розробки (T) у людино-годинах здійснюється за допомогою норми продуктивності (Productivity

Factor, PF), яка відображає часові витрати на реалізацію одного скоригованого пункту вимог:

$$T = AUCP \times PF$$

Для некомерційних або академічних проєктів, де розробка супроводжується детальним дослідженням інструментів, норма витрат часу обирається ближче до верхньої межі нормативного діапазону.

Аналіз взаємодії з веборієнтованою системою дозволив класифікувати суб'єктів системи за ступенем складності їхніх інтерфейсів доступу. Для вебплатформи опитувань виділено три типи користувачів, які взаємодіють через складний графічний веб-інтерфейс (GUI), та два типи зовнішніх інтеграцій, що здійснюють протокольну взаємодію через прикладні програмні інтерфейси (API). Детальний опис розподілу ваг акторів наведено в таблиці 2.5.1.

Таблиця 2.5.1 - Класифікація та вагові коефіцієнти суб'єктів взаємодії (UAW)

Суб'єкт (Актор)	Категорія інтерфейсу	Вага	Функціональне навантаження у системі
Респондент	Складний (Графічний інтерфейс)	3	Проходження опитувань, взаємодія з динамічною формою
Автор опитувань	Складний (Графічний інтерфейс)	3	Керування конструктором, перегляд інтерактивної аналітики
Адміністратор	Складний (Графічний інтерфейс)	3	Глобальний моніторинг системи, блокування та модерація

Сервіс III (Gemini API)	Простий (Програмний інтерфейс)	1	Автоматизоване формування структури запитань через зовнішній API
Сторонній обробник (Webhook)	Простий (Програмний інтерфейс)	1	Передача результатів анкетування у реальному часі за вказаним URL
Усього UAW		11	

Складність прецедентів використання системи оцінюється на основі кількості логічних кроків, внутрішніх розгалужень бізнес-правил та залучених таблиць бази даних під час виконання сценарію. Усі 16 варіантів використання веборієнтованого додатку розподілено на три класи складності (таблиця 2.5.2).

Таблиця 2.5.2 - Оцінка складності варіантів використання системи (UUCW)

ID	Варіант використання	Клас складності	Вага	Обґрунтування складності
UC-01	Реєстрація	Середній	2	Перевірка унікальності даних, хешування паролів, створення профілю
UC-02	Авторизація	Середній	2	Валідація облікових даних, генерація безпечних сесійних токенів
UC-03	Проходження опитування	Складний	3	Динамічний рендеринг 10 типів питань, обчислення умовної логіки
UC-04	Відновлення прогресу	Середній	2	Зчитування збережених сесійних станів, відновлення форми

UC-05	Редагування профілю	Середній	2	Оновлення текстових даних, завантаження та валідація файлів зображень
UC-06	Перегляд каталогу	Середній	2	Пагінація даних, багатокритеріальне сортування, механізм закладок
UC-07	Пошук та вибір тегів	Простий	1	Вибірка статичних категорій, лінійна фільтрація списків
UC-08	Вихід з акаунту	Простий	1	Очищення локальних даних сесії, анулювання токенів доступу
UC-09	Створення опитування	Складний	3	Ієрархічне збереження анкет, налаштування параметрів конструктора
UC-10	Генерація через ШІ	Складний	3	Асинхронний запит до Gemini API, парсинг структури, валідація JSON
UC-11	Налаштування доступу	Середній	2	Валідація списків адрес, шифрування сесійних паролів доступу
UC-12	Розширена аналітика	Складний	3	Агрегація масивів даних, розрахунок теплових карт, воронки відмов
UC-13	Перегляд відповідей	Середній	2	Структуроване відображення анкет, механізм деанонімізації
UC-14	Створення команди	Складний	3	Організація ізольованих просторів, розсилка

				запрошень, менеджмент ролей
UC-15	Модерація опитувань	Складний	3	Глобальні операції над чужими даними, примусова зміна статусів
UC-16	Керування акаунтами	Складний	3	Адміністрування користувачів, зміна системних ролей та блокування
Разом	UUCW		37	

На основі отриманих сум розраховується первинний обсяг системи:

$$UUCP = UAW + UUCW = 11 + 37 = 48$$

Таблиця 2.5.3 - Експертна оцінка чинників технічної складності (TCF)

i	Архітектурний чинник (Фактор складності)	Вага (Wi)	Оцінка (Ti)	Зважена оцінка
1	Розподілена архітектура системи	2.0	4	8.0
2	Реальний час відгуку та продуктивність	1.0	3	3.0
3	Ефективність інтерфейсу кінцевого користувача	1.0	4	4.0
4	Складність внутрішніх обчислень та обробки	1.0	3	3.0
5	Можливість повторного використання коду	1.0	2	2.0
6	Простота інсталяції та розгортання додатка	0.5	5	2.5
7	Зручність адміністрування та експлуатації	0.5	4	2.0

8	Портованість між різними платформами	2.0	4	8.0
9	Легкість внесення змін та гнучкість архітектури	1.0	4	4.0
10	Рівень паралелізму та конкурентного доступу	1.0	3	3.0
11	Спеціальні вимоги до безпеки та захисту даних	1.0	4	4.0
12	Забезпечення прямого доступу для третіх осіб	1.0	2	2.0
13	Необхідність спеціального навчання користувачів	1.0	0	0.0
Разом				41.5

Розрахунок технічного коефіцієнта: $TCF = 0.6 + (0.01 \times 41.5) = 1.015$

Оцінка чинників середовища розробки, що враховує виконання проєкту одним розробником в умовах академічного середовища, наведена в таблиці 2.5.4.

Таблиця 2.5.4 - Експертна оцінка факторів середовища розробки (EF)

j	Чинник середовища проєкту (Процес і команда)	Вага (Wj)	Оцінка (Ej)	Зважена оцінка
1	Знайомство з проєктами подібного типу	1.5	3	4.5
2	Досвід об'єктно-орієнтованого проєктування	1.0	4	4.0
3	Практичний досвід головного розробника	0.5	2	1.0

4	Рівень мотивації до успішної реалізації	1.0	5	5.0
5	Стабільність та незмінність вимог до продукту	2.0	4	8.0
6	Наявність суміщення або неповного робочого дня	-1.0	3	-3.0
7	Складність обраного інструментарію та мови	-1.0	2	-2.0
8	Можливість отримання експертної підтримки	-1.0	1	-1.0
Разом				16.5

Розрахунок коефіцієнта середовища: $EF = 1.4 + (-0.03 \times 16.5) = 0.905$.

Визначення підсумкового скоригованого обсягу системи в пунктах прецедентів:

$$AUCP = 48 \times 1.015 \times 0.905 = 44.09 \text{ пунктів}$$

Беручи до уваги використання сучасних експрес-інструментів розробки (Prisma, React Query), але зважаючи на необхідність самостійної інтеграції модулів аналітики, вебхуків та ШІ, оберемо зважене значення фактора продуктивності розробника: $PF = \frac{20 \text{ людино-годин}}{AUCP}$.

Сумарна нормативна трудомісткість розробки програмного продукту становить:

$$T = 44.09 \times 20 \approx 882 \text{ людино – години}$$

Для переведення розрахованої трудомісткості у фінансовий еквівалент використовується базова годинна ставка фахівця рівня *Middle Full-Stack Developer* в Україні, яка становить орієнтовно 680 грн/год (з урахуванням

супутніх витрат на забезпечення робочого місця, ліцензії та амортизацію обладнання).

Фінансова вартість безпосереднього етапу інженерної розробки (C) обчислюється як:

$$C = T \times 680 = 882 \times 680 = 599\,760 \text{ грн}$$

У перерахунку за поточним валютним курсом вартість створення базової версії (MVP) системи становить приблизно \$13 600 USD.

2.6. Постановка завдання на розробку програмного забезпечення

Метою розробки є створення програмного забезпечення веборієнтованої системи створення, проведення та аналізу онлайн-опитувань Survey App - відкритої платформи з сучасним TypeScript-стеком, AI-асистованою генерацією опитувань та розширеними аналітичними інструментами, що надає організаціям повний контроль над даними.

Цілі розробки:

- забезпечити авторам зручний конструктор опитувань з підтримкою 10 типів питань та умовної логіки;
- реалізувати AI-генерацію структури опитування за темою через Google Gemini 2.5 Flash API;
- розробити гнучку систему управління доступом з п'ятьма рівнями видимості;
- реалізувати розширену аналітику: NPS, теплова карта, воронка відмов, аналіз пристроїв;
- забезпечити командну роботу з розмежуванням ролей Owner/Editor/Viewer;
- реалізувати публічний каталог опитувань з пошуком та тегами;
- забезпечити безпеку даних відповідно до галузевих стандартів;

- розгорнути систему в контейнеризованому середовищі (Docker Compose + Nginx).

Задачі, що підлягають вирішенню:

- розробка REST API з 35+ ендпоінтами та автодокументацією Swagger;
- проєктування схеми бази даних PostgreSQL з 20 моделями через Prisma ORM;
- реалізація React SPA з TypeScript, Tailwind CSS та TanStack Query;
- реалізація GeminiController з промпт-інжинірингом для генерації JSON-структур опитувань;
- реалізація алгоритмів агрегації: NPS Score, теплова карта (7×24), воронка відмов;
- реалізація механізму skip logic на клієнтській стороні;
- налаштування Docker Compose для автоматизованого розгортання всіх компонентів.

о Висновки до розділу

У другому розділі виконано повний цикл розроблення вимог до програмного забезпечення веборієнтованої системи онлайн-опитувань Survey App.

Визначено та описано 16 варіантів використання системи з чотирма акторами відповідно до UML-діаграми: Неавторизований користувач (реєстрація, авторизація), Користувач (профіль, вихід, розширений перегляд), Автор (конструктор, AI-генерація, аналітика, команди, налаштування) та Адмін (модерація, управління акаунтами).

Розроблено функціональні вимоги у вигляді загальної моделі (13 модулів/контролерів) та переліку 13 функціональних вимог з трьома рівнями пріоритету. Побудовано матрицю трасування вимог.

На основі проведеного розрахунку за методикою Use Case Points (UCP) встановлено, що нормативна трудомісткість розробки веборієнтованої

системи опитувань становить 882 людино-години. Прогнозована фінансова вартість створення базової версії (MVP) продукту складає 13 600 USD. Отримані показники підтверджують економічну доцільність та високу конкурентоспроможність проєкту, оскільки розробка є кастомним, функціонально розширеним і водночас фінансово доступним рішенням порівняно з дороговартісними комерційними альтернативами.

За результатами розділу сформовано технічне завдання на розробку програмного забезпечення.

3. КОНСТРУЮВАННЯ ТА РОЗРОБЛЕННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

3.1. Архітектура програмного забезпечення

Система Survey App реалізована за архітектурним стилем клієнт-сервер (Client-Server) з монолітним бекендом та SPA-клієнтом (Single Page Application). Для опису архітектури використовується модель C4 (Context, Container, Component), що дозволяє представити систему на різних рівнях деталізації [9].

Вибір монолітної архітектури замість мікросервісної обумовлено такими міркуваннями: система є навчальним проєктом з чітко обмеженим функціоналом; монолітна архітектура значно простіша в розгортанні та налагодженні; Docker-контейнеризація забезпечує ізоляцію та можливість подальшого переходу до мікросервісів; зв'язаність компонентів бекенду (спільна БД, ORM) виправдовує єдиний застосунок.

Діаграма контексту системи визначає межі розроблюваного програмного забезпечення, його ключових користувачів (акторів) та інтеграційні зв'язки із зовнішніми сервісами, які знаходяться поза безпосереднім контролем системи, але забезпечують її розширену функціональність (рис. 3.1).

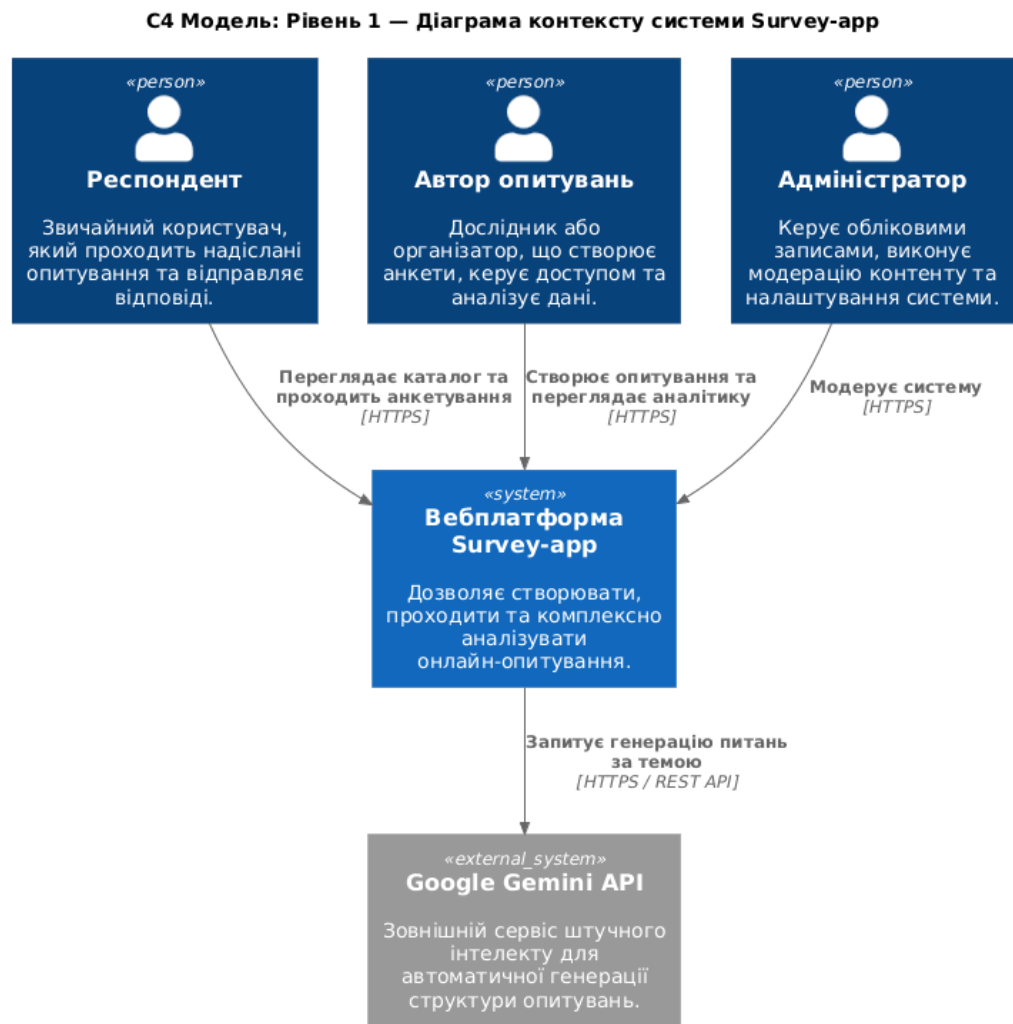


Рисунок 3.1 - Діаграма контексту (C4 model рівень 1)

Діаграма контейнерів деталізує внутрішню структуру системи, показуючи розподіл високорівневої логіки між окремими програмними та технологічними компонентами (контейнерами), що виконуються в ізольованих середовищах. Вона описує технологічний стек, обов'язки кожного контейнера та протоколи взаємодії між ними (рис. 3.2).

C4 Модель: Рівень 2 — Діаграма контейнерів системи

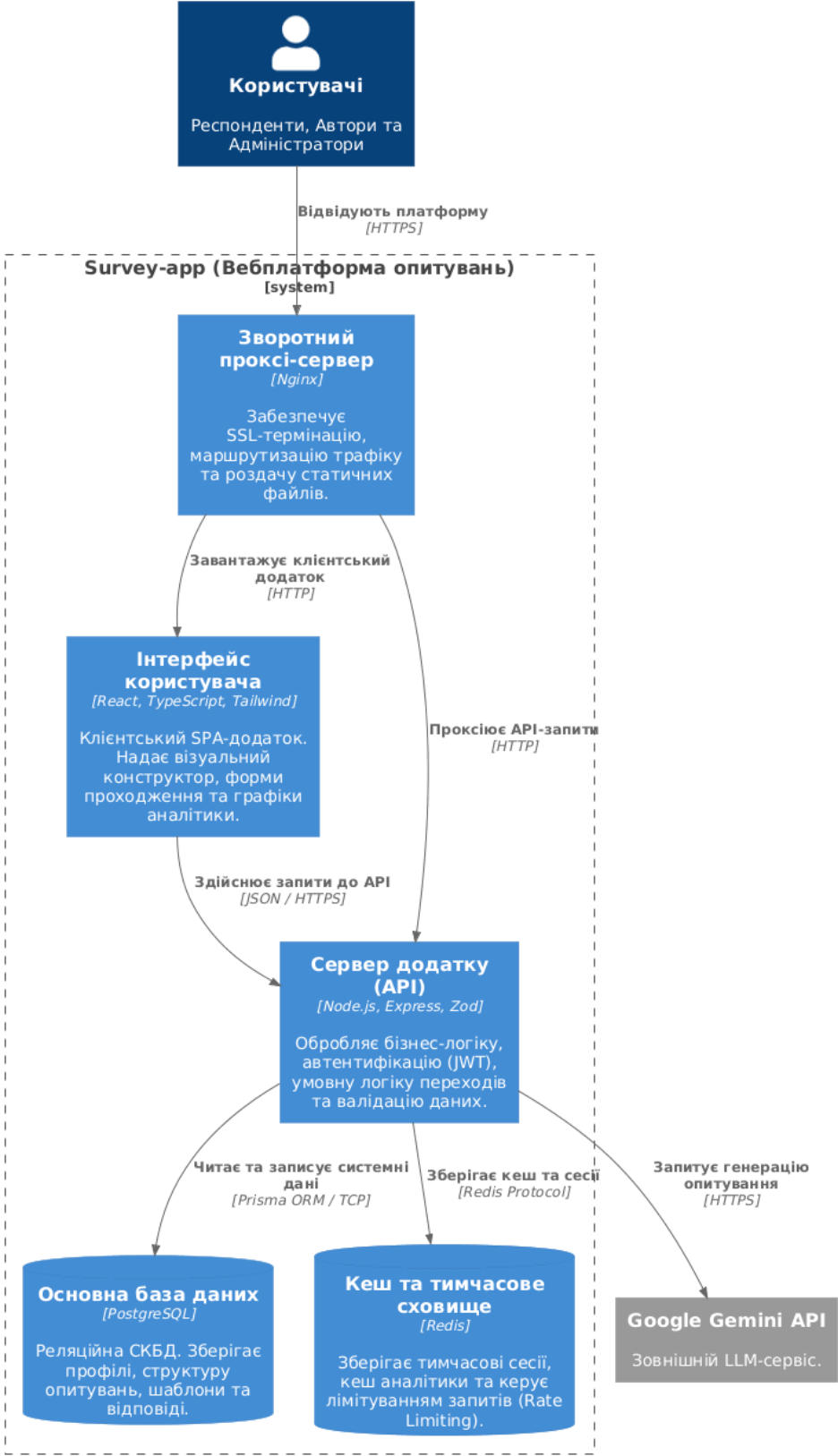


Рисунок 3.2 - Діаграма контейнерів (C4 model рівень 2)

Рівень компонентів розкриває внутрішню архітектуру серверної частини (Backend API), демонструючи, як саме реалізовано бізнес-логіку

всередині Node.js-додатку. В основі лежить шаблон проектування Controller-Service-Repository, що забезпечує низьку зв'язність коду та легкість у тестуванні (рис.3.3).

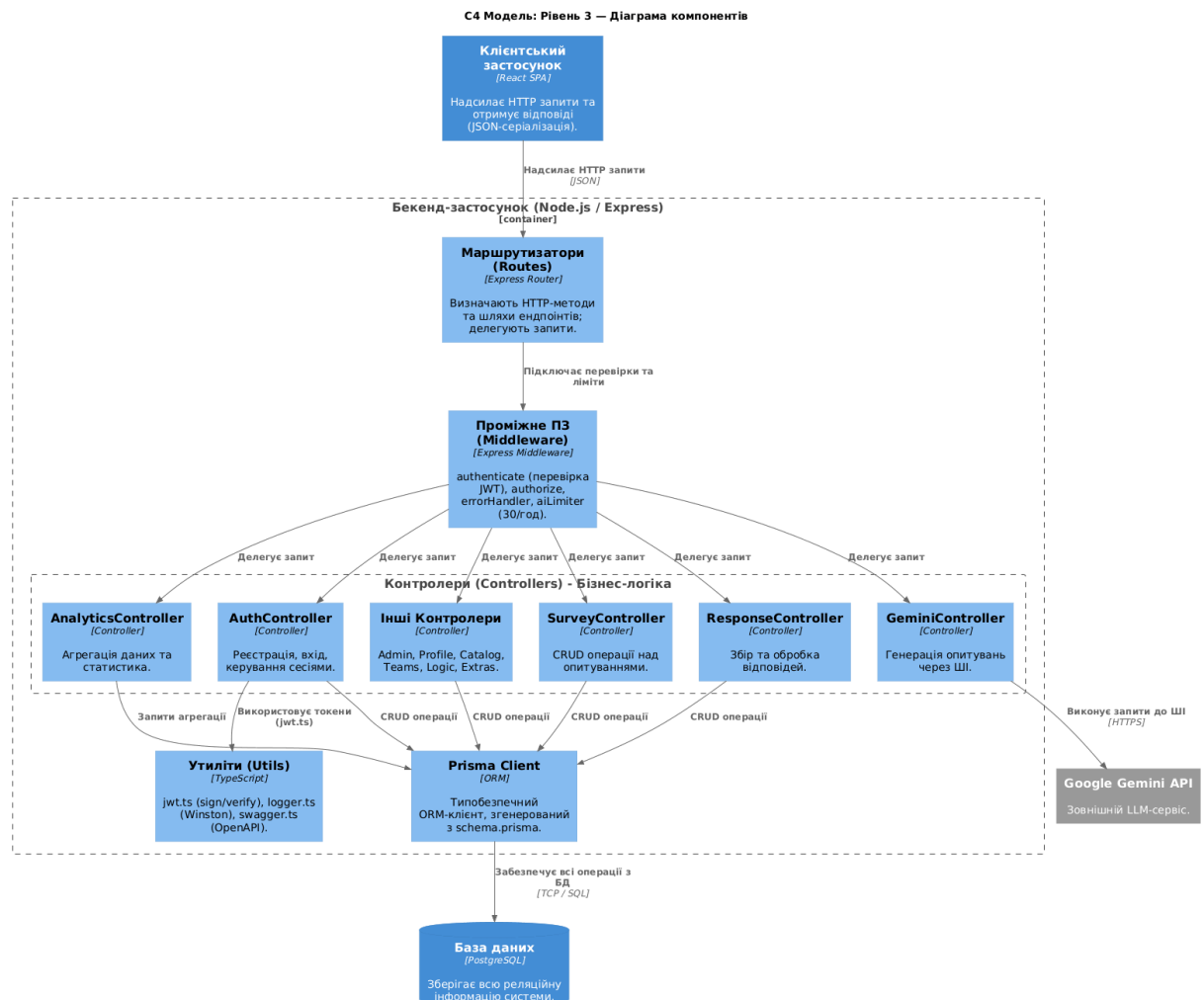


Рисунок 3.3 - Діаграма компонентів (C4 model рівень 3)

Фізичне та логічне розділення компонентів дозволяє виділити три ключові рівні: презентаційний шар (клієнтський Single Page Application на базі React [10]), проміжний шар бізнес-логіки та маршрутизації (Node.js API-сервер на базі фреймворку Express [11,12,13]) та шар управління даними (реляційна СУБД PostgreSQL у поєднанні з In-memory сховищем Redis та ORM-клієнтом Prisma [14,15,16]). На рівні серверного ядра реалізовано шаблон MVC (Model-View-Controller), де функції представлення замінено механізмом серіалізації даних у формат JSON для подальшої передачі фронтенд-клієнту.

Масштабованість системи забезпечується двома векторами. З одного боку, впровадження кешування запитів та збереження тимчасових даних проходження опитувань у сховищі Redis значно знижує навантаження на основну базу даних PostgreSQL, реалізуючи вертикальну оптимізацію продуктивності. З іншого боку, завдяки технології контейнеризації Docker та наявності зворотного проксі-сервера Nginx, архітектура підтримує горизонтальне масштабування: у разі зростання навантаження (наприклад, під час масового розсилання анкет) можна розгорнути додаткові екземпляри Node.js-контейнерів, налаштувавши Nginx як балансувальник навантаження [17,18].

3.2. Архітектурні рішення та обґрунтування вибору засобів розробки

Проектування та реалізація сучасної веборієнтованої системи створення, проведення та аналізу онлайн-опитувань вимагає застосування архітектурних підходів, які гарантують стабільну обробку нерівномірних за інтенсивністю потоків даних, безпеку збереження персональної інформації та гнучкість розширення функціоналу. Розроблена система реалізує власну підсистему автентифікації без використання сторонніх провайдерів ідентифікації, що гарантує повний контроль над конфіденційними даними користувачів. На відміну від класичних сесійних підходів, які вимагають збереження стану на сервері, у системі застосовано механізм підписаних токенів доступу (JSON Web Tokens). Це забезпечує високу масштабованість завдяки відсутності збереження стану авторизації на бекенді та знижує загальну складність реалізації. Токен містить базове інформаційне навантаження з ідентифікатором, електронною поштою та роллю користувача, має обмежений термін дії та передається у заголовках запитів. На серверному рівні спеціальне проміжне програмне забезпечення декодує токен та верифікує права доступу. Для безпечного зберігання паролів застосовано адаптивний алгоритм хешування bcrypt із високим коефіцієнтом складності, що надійно захищає систему від атак повним перебором.

Для реалізації функції інтелектуальної генерації структури опитувань інтегровано хмарну мовну модель Google Gemini 2.5 Flash через архітектурний стиль REST API. Порівняно з альтернативними рішеннями на ринку, ця модель пропонує оптимальний баланс високої швидкості генерації відповідей, відмінної якості формування структурованих об'єктів формату JSON та наявності достатніх безкоштовних лімітів для забезпечення стабільної роботи платформи. Запит до штучного інтелекту формується на серверній частині із використанням спеціально спроектованого промпту, а отримана відповідь автоматично очищується від текстової розмітки та парситься у валідний об'єкт бази даних.

Вибір технологічного стека здійснювався з огляду на суворі вимоги до продуктивності, безпеки, типізації та можливості швидкого масштабування. Клієнтська частина побудована на базі бібліотеки React та мови TypeScript для забезпечення гнучкої компонентної архітектури й суворої статичної типізації. Збирачем проєкту виступає система Vite, яка гарантує надшвидку компіляцію коду під час розробки. Візуальний дизайн реалізовано за допомогою утилітарного фреймворку Tailwind CSS, що дозволило створити легкий та адаптивний інтерфейс. Для роботи з формами та декларативної валідації використовуються інструменти React Hook Form і Zod. Взаємодія з сервером та управління асинхронним станом, включно з кешуванням і фоновими оновленнями, забезпечується зв'язкою бібліотек Axios та TanStack Query, тоді як візуалізація розширеної аналітики реалізована за допомогою легкої бібліотеки Chart.js.

Серверна частина системи базується на платформі Node.js та фреймворку Express. Вибір Express аргументується низьким порогом входу, найбільшою екосистемою готового проміжного програмного забезпечення для забезпечення безпеки, логування та документування програмних інтерфейсів. Використання мови TypeScript на обох рівнях архітектури дозволяє спільно використовувати об'єкти передачі даних між клієнтом та сервером, запобігаючи помилкам несумісності. Основним сховищем даних

виступає реляційна система управління базами даних PostgreSQL, що забезпечує надійну роботу з JSON-структурами. Для взаємодії з базою даних обрано інструмент Prisma ORM, який пропонує підхід на основі попередньо визначеної схеми, повну типобезпеку запитів, автоматичні міграції та вбудований захист від проблеми надлишкових запитів. Додатково для кешування результатів аналітики та роботи підсистеми обмеження частоти запитів інтегровано високошвидкісне сховище в оперативній пам'яті Redis.

Уся інфраструктура застосунку розгортається у відтворюваному ізолюваному середовищі за допомогою інструментів контейнеризації Docker та Docker Compose. Маршрутизацію мережевого трафіку, термінацію захищених з'єднань, резервне перенаправлення для односторінкового додатка та кешування статичних файлів забезпечує зворотний проксі-сервер Nginx. Основним середовищем розробки виступає редактор Visual Studio Code завдяки його нативній інтеграції з TypeScript, низькому споживанню ресурсів та наявності широкої екосистеми розширень. Тестування та налагодження програмних інтерфейсів здійснювалося за допомогою платформи Postman.

3.3. Конструювання програмного забезпечення

3.3.1. Опис структури бази даних ...

В якості системи управління базами даних для веборієнтованої системи онлайн-опитувань використовується реляційна СУБД PostgreSQL. База даних серверу призначена для збереження облікових записів користувачів, конфігурацій та параметрів доступу до опитувань, ієрархічної структури конструктора анкет, проміжного стану проходження, а також фінальних відповідей респондентів для подальшого аналізу.

Завдяки використанню ORM Prisma на фізичному рівні PostgreSQL суворо підтримується цілісність посилань та оптимізовано індексування полів, що часто використовуються для фільтрації та пошуку контенту. Детальний опис таблиць розробленої бази даних наведено у таблицях 3.1-3.12. Логічна модель зв'язків між сутностями наведена на рисунку 3.4.

Таблиця 3.1 - Опис таблиці users (Користувачі системи)

Назва поля	Тип даних	Опис
id	UUID	Первинний ключ (Primary Key), унікальний системний ідентифікатор користувача.
email	VARCHAR(255)	Унікальне поле (Unique), електронна адреса користувача для автентифікації.
password_hash	VARCHAR(255)	Криптографічний хеш пароля, згенерований за допомогою алгоритму bcrypt.
name	VARCHAR(255)	Повне ім'я або відображуваний нікнейм користувача в системі.
role	VARCHAR(50)	Рольова модель користувача. Дозволені значення переліку: ADMIN, CREATOR, RESPONDENT.
is_active	BOOLEAN	Прапорець активності облікового запису (використовується для блокування модераторами).
created_at	TIMESTAMP	Дата та час реєстрації облікового запису в системі.
updated_at	TIMESTAMP	Дата та час останнього внесення змін у профіль користувача.

Таблиця 3.2 - Опис таблиці user_profiles (Додаткові метадані профілів)

Назва поля	Тип даних	Опис
id	UUID	Первинний ключ таблиці профілів користувачів.

user_id	UUID	Зовнішній ключ (Foreign Key), що посиляється на users(id) з умовою каскадного видалення.
username	VARCHAR(255)	Унікальний публічний нікнейм користувача для формування персональних посилань.
bio	TEXT	Короткий текстовий опис профілю автора опитувань («про себе»).
avatar_url	VARCHAR(512)	Посилання на завантажене зображення аватара користувача в хмарному сховищі.
theme	VARCHAR(50)	Персоналізоване налаштування теми інтерфейсу панелі керування (light, dark, system).

Таблиця 3.3 - Опис таблиці teams (Командні простори для спільної роботи)

Назва поля	Тип даних	Опис
id	UUID	Первинний ключ командного простору.
owner_id	UUID	Зовнішній ключ, який визначає власника команди та посиляється на users(id).
name	VARCHAR(255)	Офіційне найменування робочого простору або організації.
slug	VARCHAR(255)	Унікальний URL-ідентифікатор команди для швидкого переходу.

Таблиця 3.4 - Опис таблиці surveys (Основні сутності опитувань)

Назва поля	Тип даних	Опис
id	UUID	Первинний ключ опитування, що використовується для генерації унікальних лінків на анкети.

user_id	UUID	Зовнішній ключ автора форми, посилається на users(id). Поле проіндексоване.
title	VARCHAR(255)	Головний заголовок анкетної форми опитування.
description	TEXT	Розширений опис, інструкція чи привітання для респондентів перед початком.
status	VARCHAR(50)	Поточний стан життєвого циклу форми: DRAFT, ACTIVE, PAUSED, CLOSED.
is_anonymous	BOOLEAN	Прапорець, що визначає, чи будуть приховуватися IP-адреси та ID респондентів.
starts_at	TIMESTAMP	Запланована дата та час автоматичного відкриття доступу до збору відповідей.
ends_at	TIMESTAMP	Запланована дата та час автоматичного закриття форми опитування.

Таблиця 3.5 - Опис таблиці survey_access (Параметри безпеки та видимості анкет)

Назва поля	Тип даних	Опис
id	UUID	Первинний ключ конфігурації доступу опитування.
survey_id	UUID	Зовнішній ключ із обмеженням Unique, посилається на surveys(id).
visibility	VARCHAR(50)	Політика приватності: PUBLIC, UNLISTED, PRIVATE, PASSWORD, EMAIL_LIST.
password_hash	VARCHAR(255)	Опціональний хеш пароля, необхідний для проходження анкети респондентом.

email_list	TEXT[]	Масив дозволених email-адрес респондентів (для корпоративних опитувань).
max_responses	INTEGER	Опціональний ліміт максимальної кількості відповідей, після якого форма закривається.

Таблиця 3.7 - Опис таблиці questions (Запитання конструктора анкет)

Назва поля	Тип даних	Опис
id	UUID	Первинний ключ сутності конкретного запитання опитування.
survey_id	UUID	Зовнішній ключ, зв'язок «один-до-багатьох» із батьківською таблицею surveys(id).
text	TEXT	Формулювання самого запитання, що відображається респонденту.
type	VARCHAR(50)	Тип поля вводу: SINGLE_CHOICE, MULTIPLE_CHOICE, TEXT, MATRIX, NPS, RATING тощо.
required	BOOLEAN	Логічний прапорець, що забороняє відправку анкети без відповіді на це питання.
order_index	INTEGER	Ціле число, що визначає точний порядок відображення запитання на сторінці форми.
settings	JSONB	Додаткові специфічні конфігурації запитання (ліміти символів, мін/макс значення).

Таблиця 3.8 - Опис таблиці question_options (Динамічні варіанти відповідей)

Назва поля	Тип даних	Опис
id	UUID	Первинний ключ варіанта відповіді.

question_id	UUID	Зовнішній ключ запиту вибору, посилається на questions(id).
text	TEXT	Текстове значення конкретного варіанта відповіді.
order_index	INTEGER	Порядковий індекс сортування варіантів у межах одного запиту.

Таблиця 3.9 - Опис таблиці skip_logic (Логіка опитування)

Назва поля	Тип даних	Опис
id	UUID	Первинний ключ правила розгалуження.
question_id	UUID	Зовнішній ключ вихідного запиту, з якого відбувається перехід; посилається на questions(id).
condition_type	VARCHAR(50)	Логічний оператор порівняння умови переходів (наприклад, equals, не дорівнює).
condition_value	VARCHAR(255)	Конкретне тригерне значення відповіді респондента, що активує перехід.
target_question_id	UUID	Системний ідентифікатор запиту, до якого система має примусово перестрибнути.

Таблиця 3.10 - Опис таблиці response_progress (Тимчасові стани автозбереження сесій)

Назва поля	Тип даних	Опис

id	UUID	Первинний ключ запису незавершеної сесії проходження.
survey_id	UUID	Зовнішній ключ опитування, посилається на surveys(id).
session_key	VARCHAR(255)	Унікальний токен тимчасової сесії, що ідентифікує конкретного респондента без авторизації.
answers_json	JSONB	Неструктурований проміжний зліпок заповнених полів форми анкетування.
current_page	INTEGER	Номер сторінки опитування, на якій зупинився респондент.
updated_at	TIMESTAMP	Час останньої автоматичної фонові синхронізації прогресу клієнта з API.

Таблиця 3.11 - Опис таблиці survey_responses (Остаточні сесії поданих анкет)

Назва поля	Тип даних	Опис
id	UUID	Первинний ключ транзакції проходження форми опитування.
survey_id	UUID	Зовнішній ключ цільового опитування, посилається на surveys(id).

user_id	UUID	Опціональний зовнішній ключ авторизованого респондента; посилається на users(id).
ip_address	VARCHAR(45)	Мережева IP-адреса клієнта (заповнюється лише якщо анкету не марковано як анонімну).
submitted_at	TIMESTAMP	Системна мітка точного часу остаточного успішного подання анкети.

Таблиця 3.12 - Опис таблиці answers (Конкретні атомарні відповіді респондентів)

Назва поля	Тип даних	Опис
id	UUID	Первинний ключ атомарного запису відповіді.
response_id	UUID	Зовнішній ключ загальної сесії проходження, посилається на survey_responses(id).
question_id	UUID	Зовнішній ключ запитання, на яке дано відповідь; посилається на questions(id).

text_value	TEXT	Опціональне поле для збереження текстових відповідей чи посилань на завантажені файли.
number_value	DOUBLE PRECISION	Опціональне поле для збереження числових відповідей (оцінки, шкали, NPS-індекси).
matrix_json	JSONB	Спеціалізована JSON-структура для збереження двовимірних відповідей матричного типу.

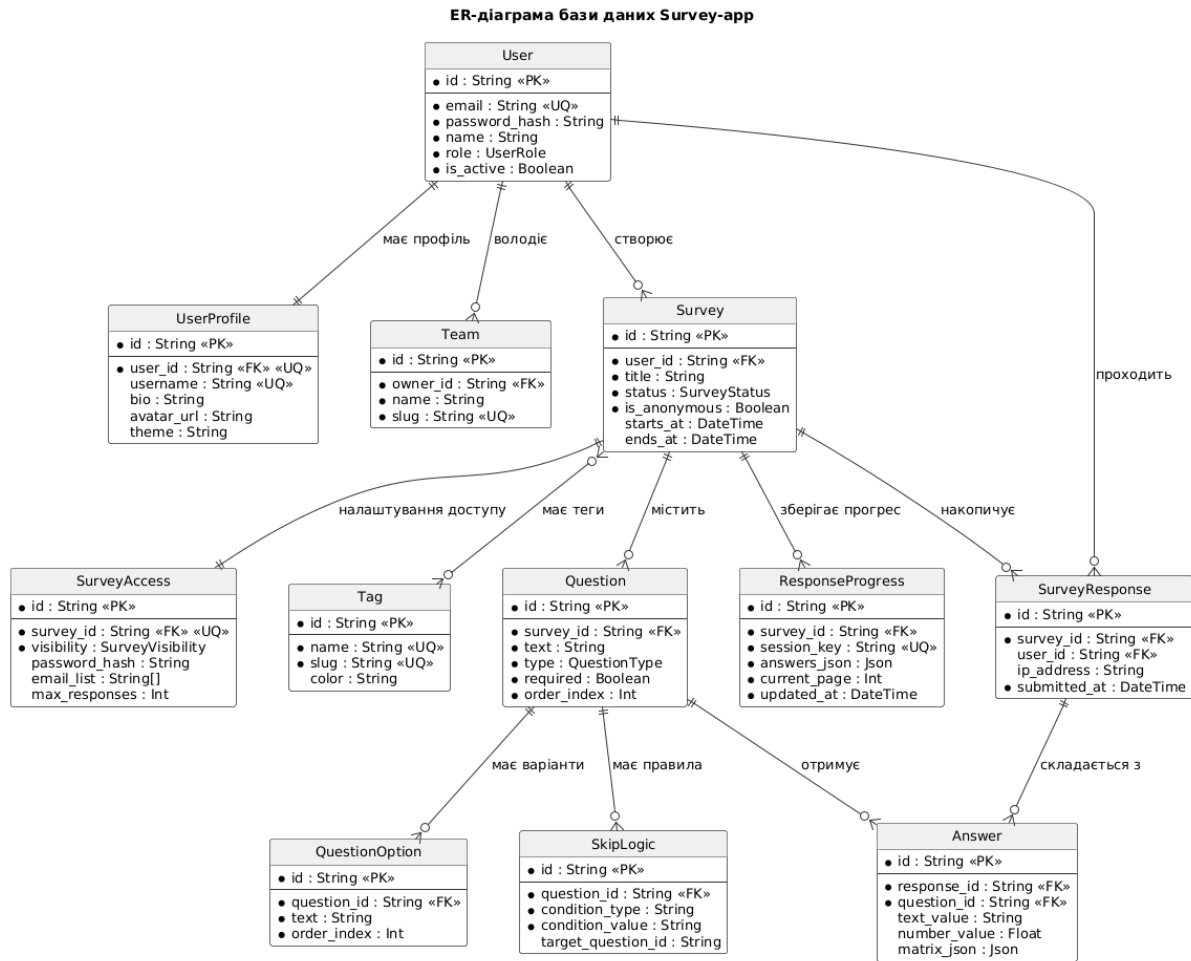


Рисунок 3.2 - ER діаграма сутностей

У процесі проєктування, написання вихідного коду, тестування та розгортання веборієнтованої системи онлайн-опитувань було задіяно низку сучасних спеціалізованих утиліт та інструментів адміністрування. Використання стандартизованого програмного забезпечення дозволило прискорити процес розробки, забезпечити відтворюваність середовища та спростити налагодження клієнт-серверної взаємодії.

Опис основних утиліт та стороннього програмного забезпечення, що використовувалося у розробці, наведено в таблиці 3.13.

Таблиця 3.13 - Опис утиліт та засобів розробки

№ п/п	Назва утиліти	Опис застосування
1	Visual Studio Code	Головне інтегроване середовище розробки (IDE). Використовувалось для написання коду клієнтської та серверної частин завдяки нативній підтримці TypeScript, інтеграції з Git та розширенням для аналізу коду.
2	Postman	Програмна платформа для тестування REST API ендпоінтів. Використовувалась для імітації клієнтських HTTP-запитів, перевірки JWT-автентифікації та налагодження контролерів бекенду без участі фронтенду.
3	Prisma Studio	Вбудований графічний веб-інтерфейс (GUI) екосистеми Prisma. Застосовувався для швидкого візуального перегляду, фільтрації та редагування записів безпосередньо у базі даних PostgreSQL.
4	PostgreSQL	Система керування базою даних
5	Docker Desktop	Система управління контейнеризацією. Застосовувалась для локального запуску, оркестрації та управління ізольованими Docker-контейнерами і мережами (PostgreSQL, Redis, Nginx, Node.js).
6	draw.io	Інструмент для візуального моделювання. Використовувався на етапі проєктування для побудови UML-діаграм, BPMN-моделей бізнес-процесів та архітектурних C4-діаграм.
7	Swagger	Інструментарій для автоматичної генерації інтерактивної документації. Застосовувався для створення специфікації OpenAPI 3.0 на основі JSDoc-коментарів у коді бекенду.

8	Zod	Утиліта для декларативної валідації даних. Забезпечує сувору перевірку вхідних даних HTTP-запитів (тіло, параметри) на сервері та автоматичну генерацію TypeScript-типів.
9	TanStack React Query	Утиліта для управління асинхронним серверним станом на фронтенді. Забезпечує автоматичне кешування запитів, фоновий рефетч (оновлення) даних та виконання мутацій.
10	Git	Розподілена система управління версіями. Використовувалась для локальної фіксації історії змін у вихідному коді та синхронізації з віддаленим репозиторієм на платформі GitHub.
11	bcryptjs	Хешування паролів
12	jsonwebtoken	Генерація та верифікація JWT-токенів
13	axios	HTTP-клієнт для React
14	chartjs	Canvas-рендеринг: Bar, Pie діаграми
15	helmet	Встановлення захисних HTTP-заголовків

Тексти програмного коду наведені в окремому документі «Текст програми».

3.4. Аналіз безпеки даних

Управління вразливостями (Vulnerability Management) [20].
Фундаментальною основою безпеки даних у розробленій системі онлайн-опитувань є комплексний підхід до захисту конфіденційної інформації респондентів та інтелектуальної власності авторів анкет. Оскільки платформа безперервно обробляє облікові записи, ієрархічні структури опитувань та масиви зібраних відповідей, процедура управління вразливостями виходить за межі базового захисту від класичних атак на кшталт SQL-ін'єкцій. Вона охоплює сувору декларативну валідацію всього вхідного трафіку на

серверному рівні за допомогою утиліти Zod [21], що превентивно унеможливорює потрапляння шкідливого коду або некоректних типів даних до бази даних PostgreSQL. Окремим вектором моделювання загроз у цьому напрямку виступає безпека архітектурної інтеграції з хмарною мовною моделлю штучного інтелекту Google Gemini. Для запобігання компрометації криптографічних ключів усі звернення до зовнішнього API виконуються виключно на серверному рівні Node.js, без жодного експонування облікових даних клієнтському додатку. З метою нівелювання ризиків ін'єкцій у промпти з боку зломисників, користувацький ввід (тема опитування) жорстко обмежується за довжиною та проходить попередню санітарну обробку перед формуванням системного запиту до ШІ. Згенерована мовною моделлю відповідь не довіряється системі автоматично, а проходить обов'язковий структурний парсинг. Додатковим ешелonom захисту виступає ізольований лімітувальник частоти запитів для інтелектуальних маршрутів, який запобігає вичерпання квот постачальника внаслідок цілеспрямованих автоматизованих атак.

Тестування безпеки (Security Testing). Під час тестування безпеки програмного продукту особлива увага приділялася верифікації механізмів авторизації, побудованих на базі алгоритму хешування bcrypt та підписаних JWT-токенів. Було проведено перевірки стійкості програмного інтерфейсу до спроб підробки підписів, а також тестування сценаріїв горизонтального та вертикального підвищення привілеїв. Спеціалізоване серверне проміжне програмне забезпечення надійно блокує спроби неавторизованих респондентів отримати доступ до панелі керування або змінити конфігурацію чужих опитувань. Крім того, підтверджено коректність роботи захисних HTTP-заголовків, згенерованих модулем Helmet [22] та налаштованих на рівні зворотного проксі-сервера Nginx, що мінімізують ризики міжсайтового скриптингу (XSS) та запобігають атакам типу клікджекінг.

Безпека контейнерів та хмарних середовищ (Security for Container and Cloud). Управління конфігураційними секретами реалізовано через ізольовані змінні середовища, які суворо виключені із систем контролю версій, що унеможлиблює їх витік. Безпека інфраструктури посилюється розгортанням усіх компонентів у закритій віртуальній мережі Docker-контейнерів, де сервіси взаємодіють між собою без прямого доступу ззовні. У підсумку, спроектований безпековий контур платформи гарантує конфіденційність сесій, надійну ізоляцію користувацьких ролей та безпечну експлуатацію сторонніх сервісів у межах масштабованого середовища.

о Висновки до розділу

У даному розділі здійснено комплексне проєктування архітектури веборієнтованої системи онлайн-опитувань за моделлю клієнт-сервер. Структуру системи деталізовано на рівнях контексту, контейнерів і компонентів за допомогою нотації C4. Обґрунтовано вибір сучасного технологічного стека: презентаційний шар побудовано на базі React, серверну бізнес-логіку - на Node.js (Express), а управління даними реалізовано через СУБД PostgreSQL із застосуванням Prisma ORM та кеш-пам'яті Redis. Гнучкість та масштабованість інфраструктури забезпечується контейнеризацією Docker та зворотним проксі-сервером Nginx.

У межах конструювання програмного забезпечення спроектовано логічну та фізичну моделі реляційної бази даних. Структура охоплює оптимізовані таблиці для управління обліковими записами, командними просторами, ієрархією конструктора анкет, динамічною умовною логікою переходів та збором відповідей із підтримкою проміжного автозбереження прогресу респондентів. Додатково сформовано стандартизований набір утиліт (VS Code, Postman, Zod, Swagger) для оптимізації процесу розробки.

Надійний захист даних реалізовано через Stateless-автентифікацію (JWT, bcrypt), сувору серверну валідацію вхідного трафіку (Zod),

налаштування захисних HTTP-заголовків (Helmet) та мережеву ізоляцію контейнерів. Окремо розроблено безпечну серверну модель інтеграції штучного інтелекту Google Gemini, яка гарантує приховування криптографічних ключів, захист від ін'єкцій у промпти та лімітування частоти запитів. Підсумовуючи, виконаний комплекс робіт закладає надійний, масштабований та захищений фундамент для успішної програмної реалізації та розгортання платформи.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Google Forms: Free Online Form Creator. Google Workspace.
URL: <https://www.google.com/forms/about/> (дата звернення: 11.05.2026).
2. SurveyMonkey: The World's Most Popular Free Online Survey Tool. Momentive Inc.
URL: <https://www.surveymonkey.com/> (дата звернення: 11.05.2026).
3. Typeform: People-Friendly Forms and Surveys. Typeform SL.
URL: <https://www.typeform.com/> (дата звернення: 11.05.2026).
4. LimeSurvey: The Online Survey Tool. LimeSurvey GmbH. URL: <https://www.limesurvey.org/> (дата звернення: 11.05.2026).
5. Jones M., Bradley J., Sakimura N. JSON Web Token (JWT). RFC 7519. Internet Engineering Task Force (IETF), 2015.
URL: <https://datatracker.ietf.org/doc/html/rfc7519> (дата звернення: 11.05.2026).
6. Business Process Model and Notation (BPMN), Version 2.0.2. Object Management Group (OMG), 2013.
URL: <https://www.omg.org/spec/BPMN/2.0.2/> (дата звернення: 11.05.2026).
7. Silver B. BPMN Method and Style. 2nd ed. Aptos: Cody-Cassidy Press, 2011. 286 p.
8. Cockburn A. Writing Effective Use Cases. Boston: Addison-Wesley Professional, 2000. 270 p.
9. Brown S. Software Architecture for Developers: Visualise, document and explore your software architecture. Leanpub, 2018. 180 p.
10. React Documentation: official web resource for React library.
URL: <https://react.dev> (дата звернення: 11.05.2026).

11. TypeScript Documentation: official handbook and reference.
URL: <https://www.typescriptlang.org> (дата звернення: 11.05.2026).
12. Node.js v20.x Documentation: official runtime environment reference.
URL: <https://nodejs.org> (дата звернення: 11.05.2026).
13. Express.js: Node.js web application framework documentation.
URL: <https://expressjs.com> (дата звернення: 11.05.2026).
14. PostgreSQL 16 Documentation / The PostgreSQL Global Development Group. 2023.
URL: <https://www.postgresql.org/docs/16/> (дата звернення: 11.05.2026).
15. Prisma ORM Documentation: official guides and reference.
URL: <https://www.prisma.io/docs> (дата звернення: 11.05.2026).
16. Redis Documentation: official technical reference.
URL: <https://redis.io/docs> (дата звернення: 11.05.2026).
17. Docker Documentation: official reference for Docker and Docker Compose.
URL: <https://docs.docker.com> (дата звернення: 11.05.2026).
18. Nginx Documentation: official deployment and configuration guides.
URL: <https://nginx.org/en/docs/> (дата звернення: 11.05.2026).
19. Google Gemini API Documentation / Google AI for Developers. 2024.
URL: <https://ai.google.dev/gemini-api/docs> (дата звернення: 11.05.2026).
20. OWASP Top 10:2021. The Core Team. Open Web Application Security Project, 2021.
URL: <https://owasp.org/Top10/> (дата звернення: 11.05.2026).
21. Zod: TypeScript-first schema validation with static type inference.

URL: <https://zod.dev> (дата звернення: 11.05.2026).

22. Helmet: Secure Express apps by setting various HTTP headers.

URL: <https://helmetjs.github.io> (дата звернення: 11.05.2026).

– ДОДАТКИ